

API Testing Fundamentals

Understanding Web Applications

- ***What is a web application?***

A web application is a software program that runs in a web browser and communicates with a web server to perform tasks such as displaying data, submitting forms, and interacting with databases.

Client-Server Architecture:

- Web applications operate using a client-server architecture. This model involves two key parts:
 - Client: The front end, which is the web browser or app that users interact with.
 - Server: The back end, which handles the processing, data storage, and business logic."

- **Client (Front End):**

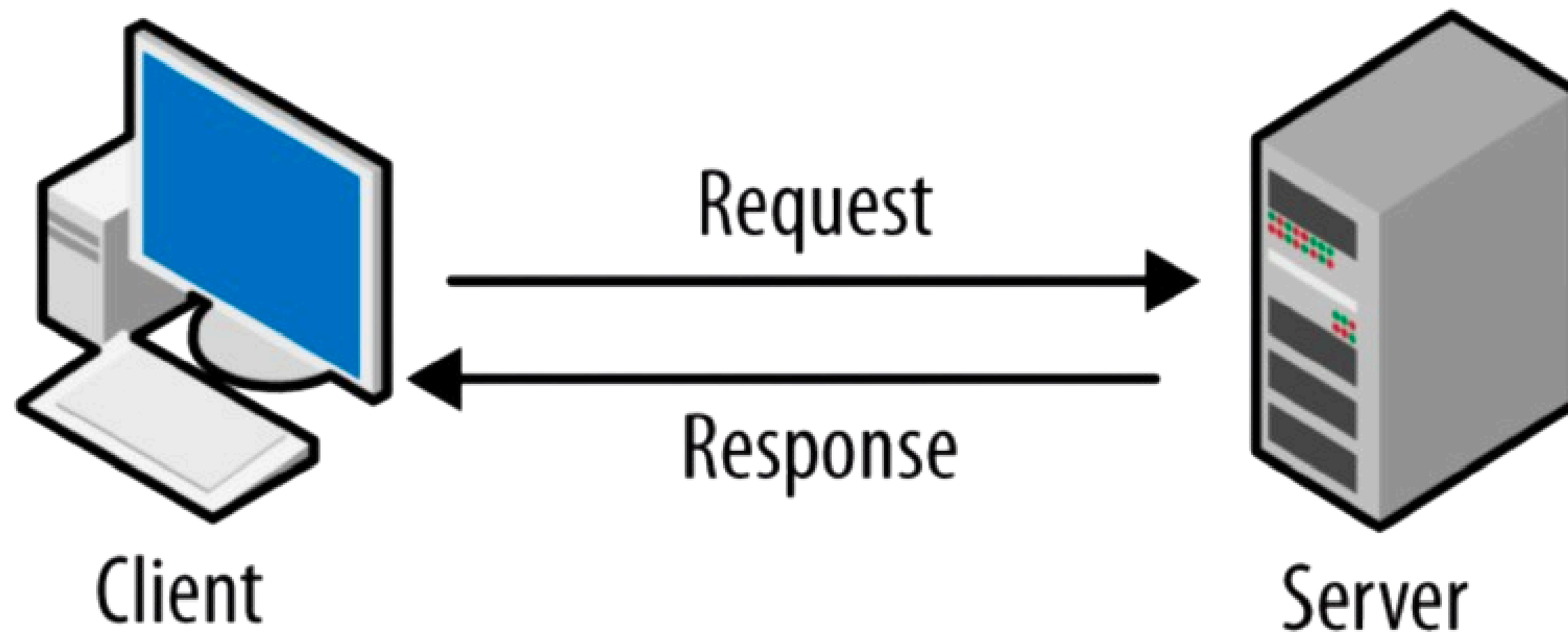
- The part of the web application that users see and interact with.
- It sends requests to the server for data or to perform actions (e.g., submitting a form).
- Example: When you visit a website like Facebook, your browser is the client.

- **Server (Back End):**

- The part that handles all processing, data management, and business logic.
- It responds to the client's requests by providing data or performing actions.
- Example: Facebook's servers store and manage your profile, posts, and interactions.

- **Communication:**

- The client and server communicate over the internet using protocols like **HTTP** (*will be mentioned in the next section*). The client sends requests and the server sends responses.



Client-Server Workflow (optional example):

- Step 1: User interacts with the web application through the browser (client) (e.g., clicks "Submit" on a form).
- Step 2: The browser sends a request to the server (e.g., to save form data).
- Step 3: The server processes the request, saves the data, and sends a response back to the browser (e.g., a confirmation message).
- Step 4: The browser displays the response (e.g., "Thank you for submitting your form").

HTTP

Hyper Text Transfer protocol

What is HTTP

- Definition:
 - HTTP (HyperText Transfer Protocol) is the protocol used for communication between web browsers (clients) and servers over the internet.

Communication between client computers and web servers is done by sending **HTTP Requests** and receiving **HTTP Responses**

What is a Protocol?

- **Definition:**

- A protocol is a set of rules or guidelines that define how data is transmitted and received across a network.

Think of it as a language that computers use to communicate with each other.

- **Key Points:**

- Just like humans need a common language to communicate, computers need protocols to exchange data.
- Protocols define how data should be formatted, transmitted, and processed so both the sender and receiver understand each other.

- **Example:** In HTTP, the browser (client) and server use a specific set of rules (HTTP protocol) to request and send web pages, images, videos, etc.

*Think of a protocol like a set of traffic rules:
Everyone driving (data transmission) must follow the same
rules (protocol) to avoid accidents (errors or failed
communication).*

What Happens When You Type nezamacademy.com? (Simplified)

1. Browser sends an HTTP request:

- When you type "nezamacademy.com" in your browser, the browser sends a GET request (will become clearer in a few mins!) to the server asking for the homepage.

2. Server processes the request:

- The server receives the request, processes it, and prepares a response.

3. Server responds:

- The server sends back an HTTP response containing the requested page (HTML, CSS, JavaScript).

4. Browser renders the page:

- The browser interprets the response and displays the LinkedIn homepage for the user to interact with.

Common Internet Protocols

FTP (File Transfer Protocol):

- Purpose: Used for transferring files between a client and a server.
- Example: Uploading files to a website's server.

SMTP (Simple Mail Transfer Protocol):

- Purpose: Used to send emails between mail servers.
- Example: Sending an email from a client like Gmail.

DNS (Domain Name System):

- Purpose: Translates domain names (like google.com) into IP addresses.
- Example: When you type a domain in the browser, DNS resolves it to the appropriate IP address.

- **TCP/IP (Transmission Control Protocol/Internet Protocol):**
 - Purpose: A suite of communication protocols used to interconnect network devices on the internet.
 - Example: Ensures data packets are delivered reliably between devices.
- **WebSocket:**
 - Purpose: Enables full-duplex communication between a client and a server over a single, long-lived connection.
 - Example: Used in chat applications or real-time notifications.

Note: These are just some of the many protocols that help the internet function smoothly.

Anatomy of an HTTP Request & Response

HTTP Request:

- The client (browser or API client) sends an HTTP request to the server to retrieve data or perform an action.

Key components of an HTTP request:

1. **Request Line** (Method, Path, HTTP Version)
2. **Headers** (Metadata about the request)
3. **Body** (Optional, data sent to the server, e.g., for POST requests)

HTTP Response:

- The server responds with an HTTP response that includes the status of the request and the requested data.

Key components of an HTTP response:

1. **Status Line** (Status code, HTTP Version)
2. **Headers** (Metadata about the response)
3. **Body** (Content sent back to the client)

Request

```
GET /posts/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
User-Agent: PostmanRuntime/7.28.4
Accept: application/json
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
Date: Sun, 07 Oct 2024 12:00:00 GMT
Content-Type: application/json;
charset=utf-8
Content-Length: 292
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: *

{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere
repellat provident occaecati
excepturi optio reprehenderit",
  "body": "quia et
suscipit\nsuscipit..."
}
```

HTTP Request – Request Line

1. Request Line:

- The request line consists of three parts:
 - HTTP Method: The action to be performed (e.g., GET, POST, PUT, DELETE).
 - URL or Path: The resource being requested (e.g., /index.html).
 - HTTP Version: Specifies the HTTP protocol version (e.g., HTTP/1.1).

HTTP Request – Request Line

Example:

GET /index.html HTTP/1.1

Explanation:

- GET: Method used to request data.
- /index.html: The path of the resource.
- HTTP/1.1: The HTTP version being used.

HTTP Request – Headers

1. Request Headers:

- Headers contain metadata about the request, such as:
 - Host: The domain or IP of the server.
 - User-Agent: Information about the client (browser) making the request.
 - Accept: Specifies the type of data the client can handle (e.g., text/html, application/json).

HTTP Request – Headers

Example:

Host: example.com

User-Agent: Mozilla/5.0

Accept: text/html

Explanation:

- Host: Tells the server the domain of the request.
- User-Agent: Describes the browser or client making the request.
- Accept: Informs the server what content type is acceptable in the response.

HTTP Request – Body

1. Request Body (optional):

- Used when sending data to the server, typically in **POST** or **PUT** requests.
- Can include **form data**, **JSON**, or other types of content.

Example (POST request):

```
POST /submit-form HTTP/1.1
```

```
Host: example.com
```

```
Content-Type: application/x-www-form-urlencoded
```

```
name=John&email=john@example.com
```

Explanation:

- Content-Type: Specifies the format of the body data.
- Body: Contains the actual data being submitted.

HTTP Response – Status Line

1. Status Line:

- The status line contains:
 - HTTP Version: Version of the HTTP protocol used.
 - Status Code: Indicates the result of the request (e.g., 200 for success, 404 for not found).
 - Reason Phrase: A brief description of the status code (e.g., "OK", "Not Found").

Example:

HTTP/1.1 200 OK

Explanation:

- 200: Success, the requested resource is available.
- OK: Reason phrase explaining the status.

HTTP Response – Headers

1. Response Headers:

- Like request headers, response headers contain metadata about the response.
 - Content-Type: Specifies the format of the returned content (e.g., text/html, application/json).
 - Content-Length: Size of the response body.
 - Date: Date and time when the response was sent.

Example:

Content-Type: text/html

Content-Length: 1234

HTTP Response – Body

1. Response Body:

- The body contains the content sent back by the server, which could be:
 - HTML for a webpage.
 - JSON data for APIs.
 - Image, video, or other binary data.

HTTP Response – Body

Example (HTML response body):

```
<html>  
  <body>  
    <h1>Welcome to Example</h1>  
  </body>  
</html>
```

Explanation:

- In this case, the response body is an HTML document, which the browser will render as a webpage.

HTTP Methods

- **GET:**

- Description: Retrieve data from the server.
- Use Case: Fetching user information, articles, etc.
- Example: GET /api/users

- **POST:**

- Description: Send data to the server to create a new resource.
- Use Case: Submitting a form, creating new users.
- Example: POST /api/users with a JSON body containing user details.

- **PUT:**

- Description: Update an existing resource on the server.
- Use Case: Modifying user details.
- Example: PUT /api/users/1 with a JSON body for updated user details.

- **PATCH:**

- Description: Partially update an existing resource.
- Use Case: Changing only specific fields in a resource.
- Example: PATCH /api/users/1 to update just the email address.

- **DELETE:**

- Description: Remove a resource from the server.
- Use Case: Deleting a user account.
- Example: DELETE /api/users/1

HTTP Status Codes

Important HTTP Status Codes for API Testing

- **2xx Success Codes:**

- 200 OK: Successful request.
- 201 Created: Resource successfully created.
- 204 No Content: Successful request with no content to return.

- **4xx Client Errors:**

- 400 Bad Request: Invalid request syntax.
- 401 Unauthorized: Authentication required.
- 403 Forbidden: Access denied.
- 404 Not Found: Resource not found.

- **5xx Server Errors:**

- 500 Internal Server Error: Server encountered an error.
- 503 Service Unavailable: Service is temporarily unavailable.

HTTP Methods

Read -> Get

Retrieve data from the server

Create -> Post

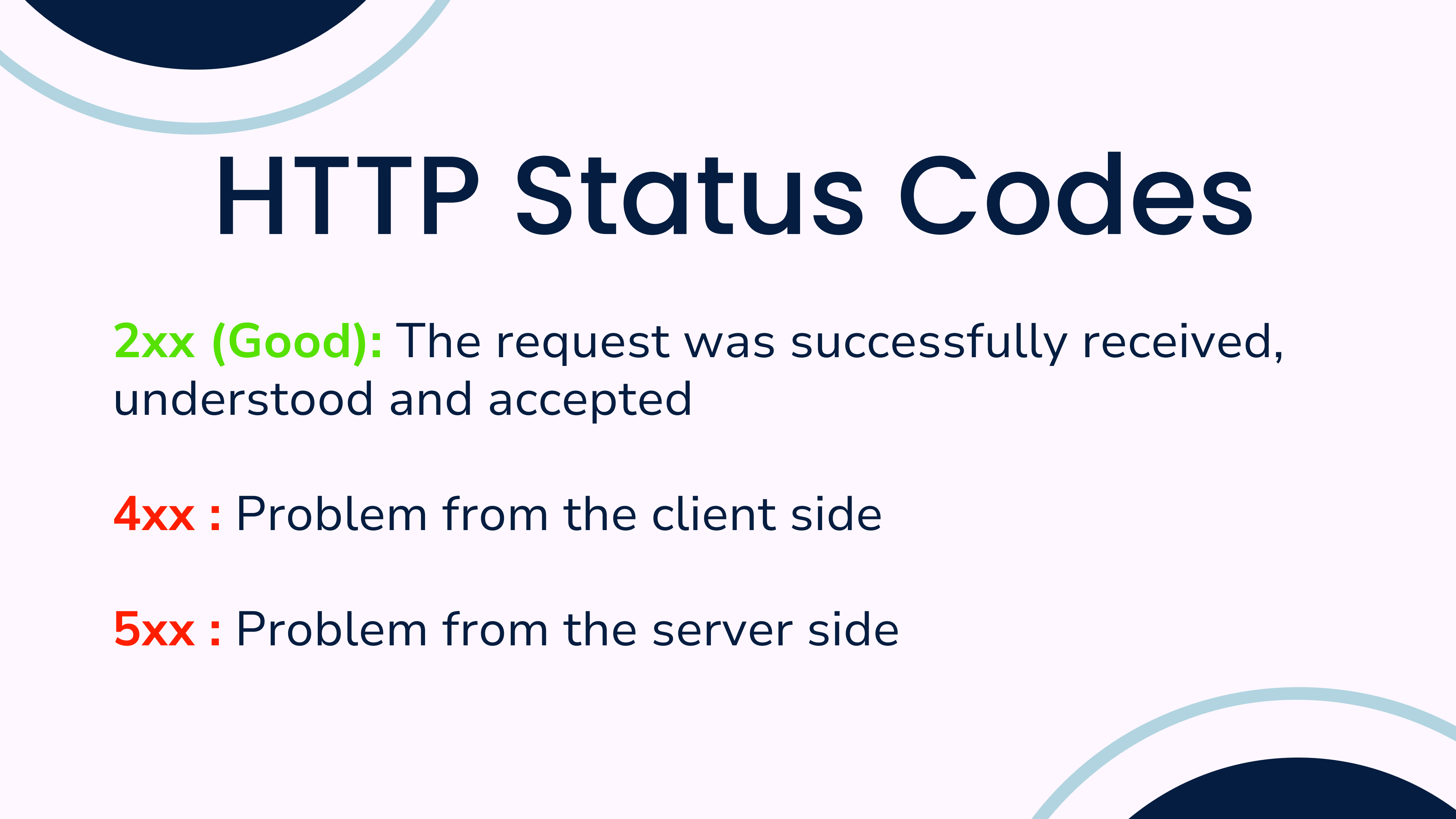
Submit data from the server

Update -> Put

Update data already on the server

Delete -> Delete

Delete data from the server

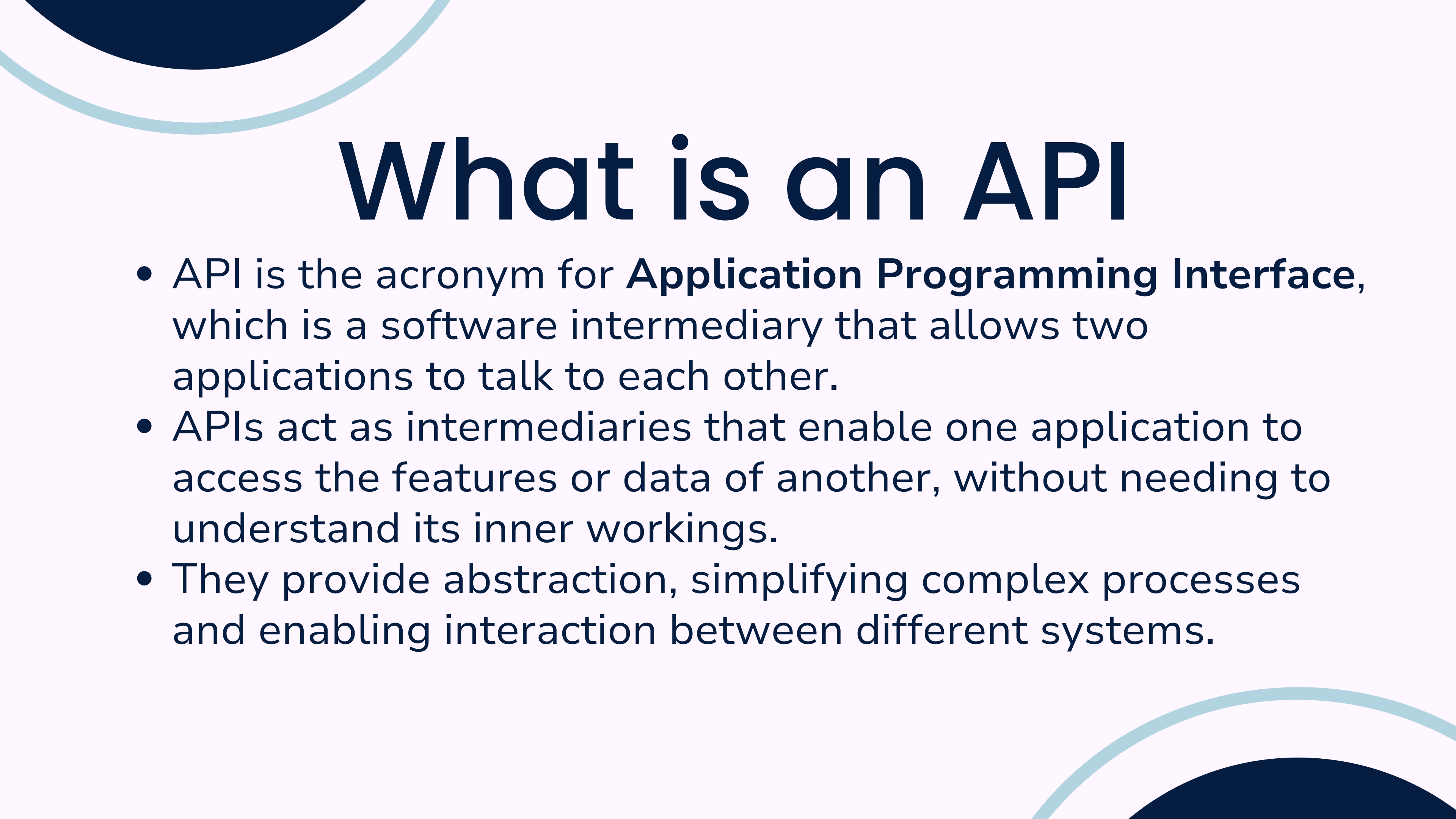


HTTP Status Codes

2xx (Good): The request was successfully received, understood and accepted

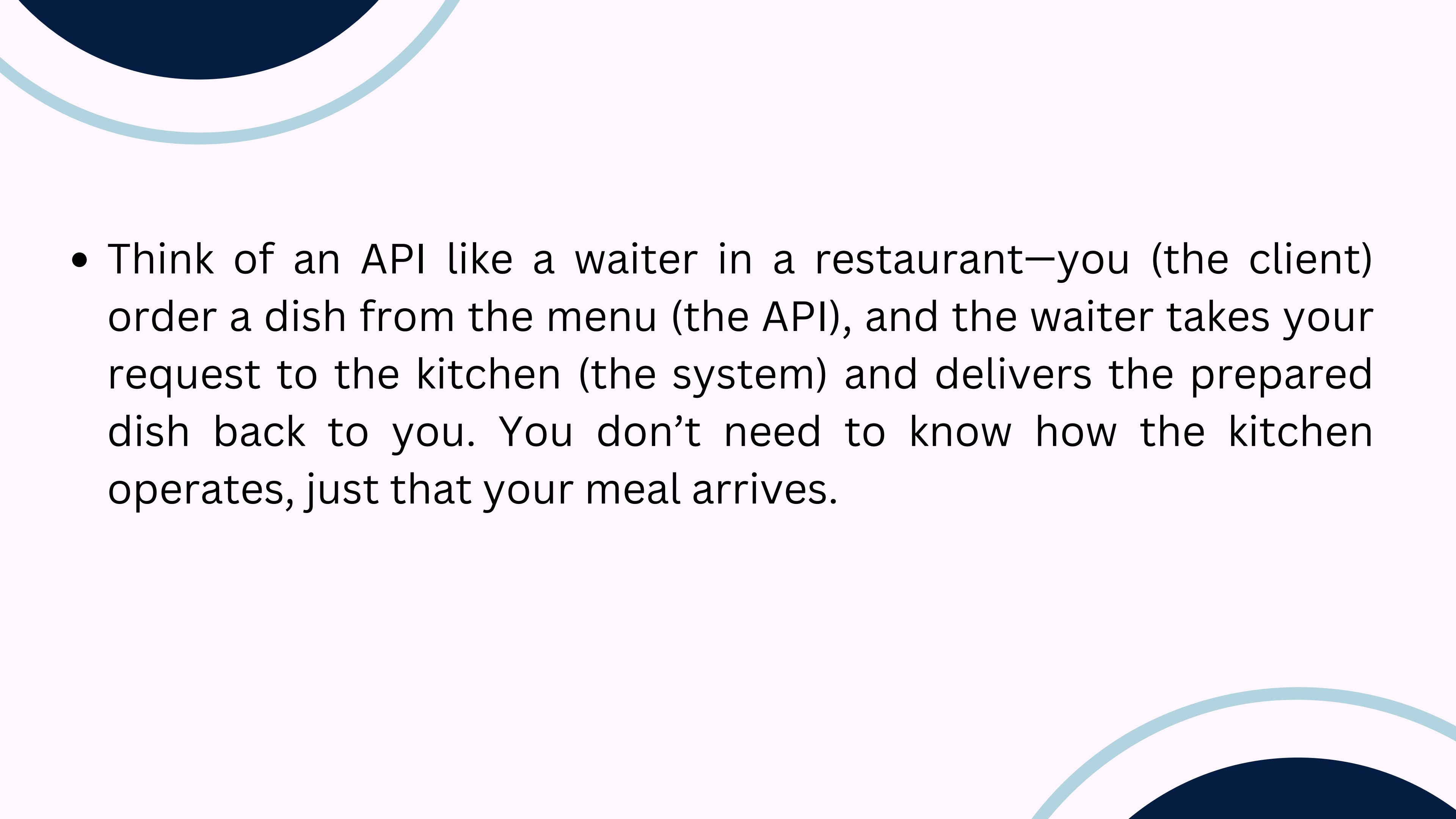
4xx : Problem from the client side

5xx : Problem from the server side



What is an API

- API is the acronym for **Application Programming Interface**, which is a software intermediary that allows two applications to talk to each other.
- APIs act as intermediaries that enable one application to access the features or data of another, without needing to understand its inner workings.
- They provide abstraction, simplifying complex processes and enabling interaction between different systems.

- 
- Think of an API like a waiter in a restaurant—you (the client) order a dish from the menu (the API), and the waiter takes your request to the kitchen (the system) and delivers the prepared dish back to you. You don't need to know how the kitchen operates, just that your meal arrives.



Client

Waiter/API

Chef/Server

Common Types of APIs

Local APIs:

- These APIs enable communication within a single system or between components of an application.
- Example: Operating System APIs, like Windows APIs, which allow applications to interact with the underlying operating system to manage tasks like file access or memory management.

Web APIs:

- Designed for communication between different applications over the internet.
 - These APIs use HTTP protocols to send and receive data, and are commonly used to integrate web services or apps.
- 

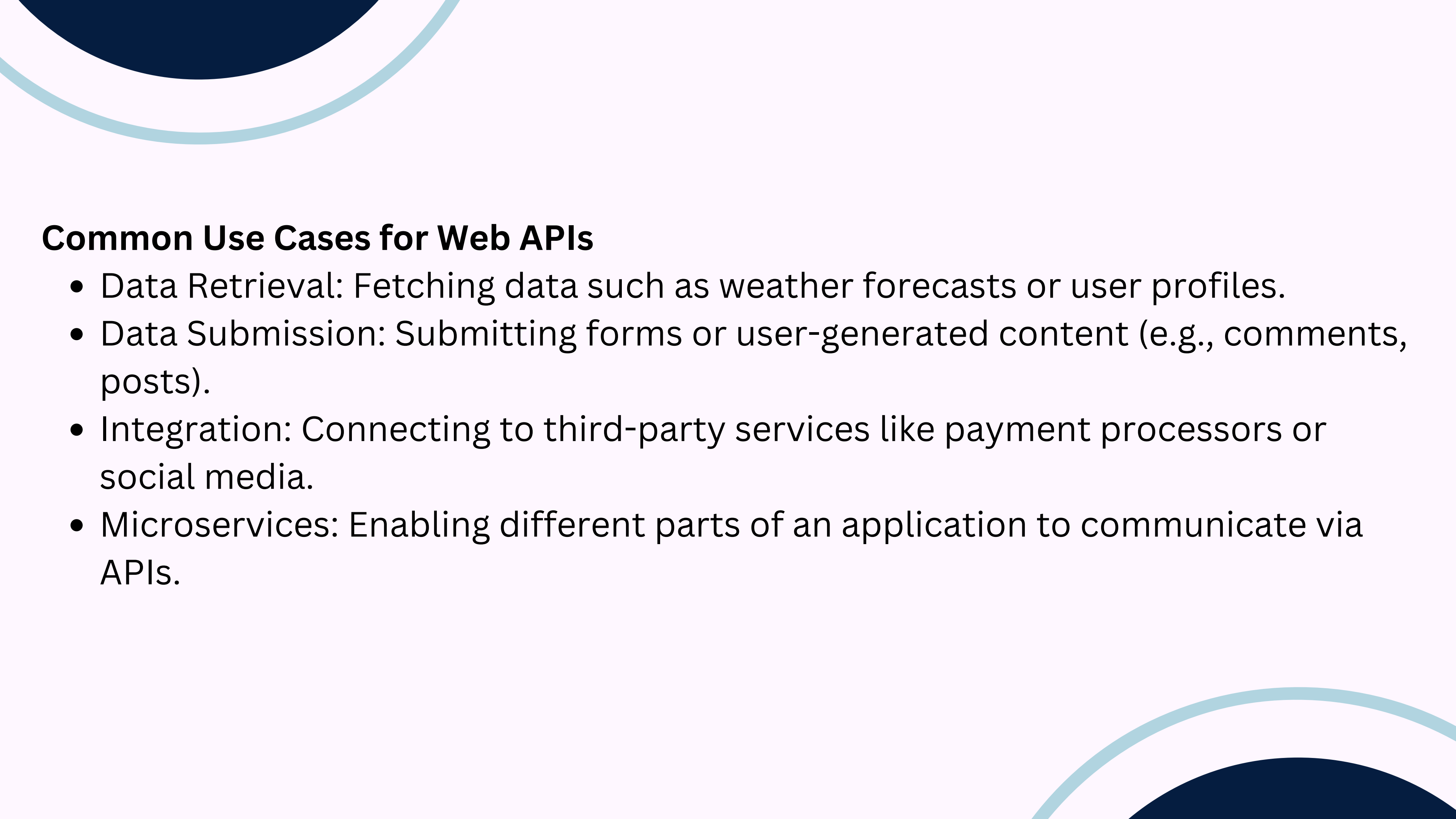
Library APIs:

- Provided by software libraries, these APIs offer ready-made functions that other programs can use.
- Example: A graphics library API enables a developer to use pre-built functions to create shapes or animations without writing the code from scratch.

Hardware APIs:

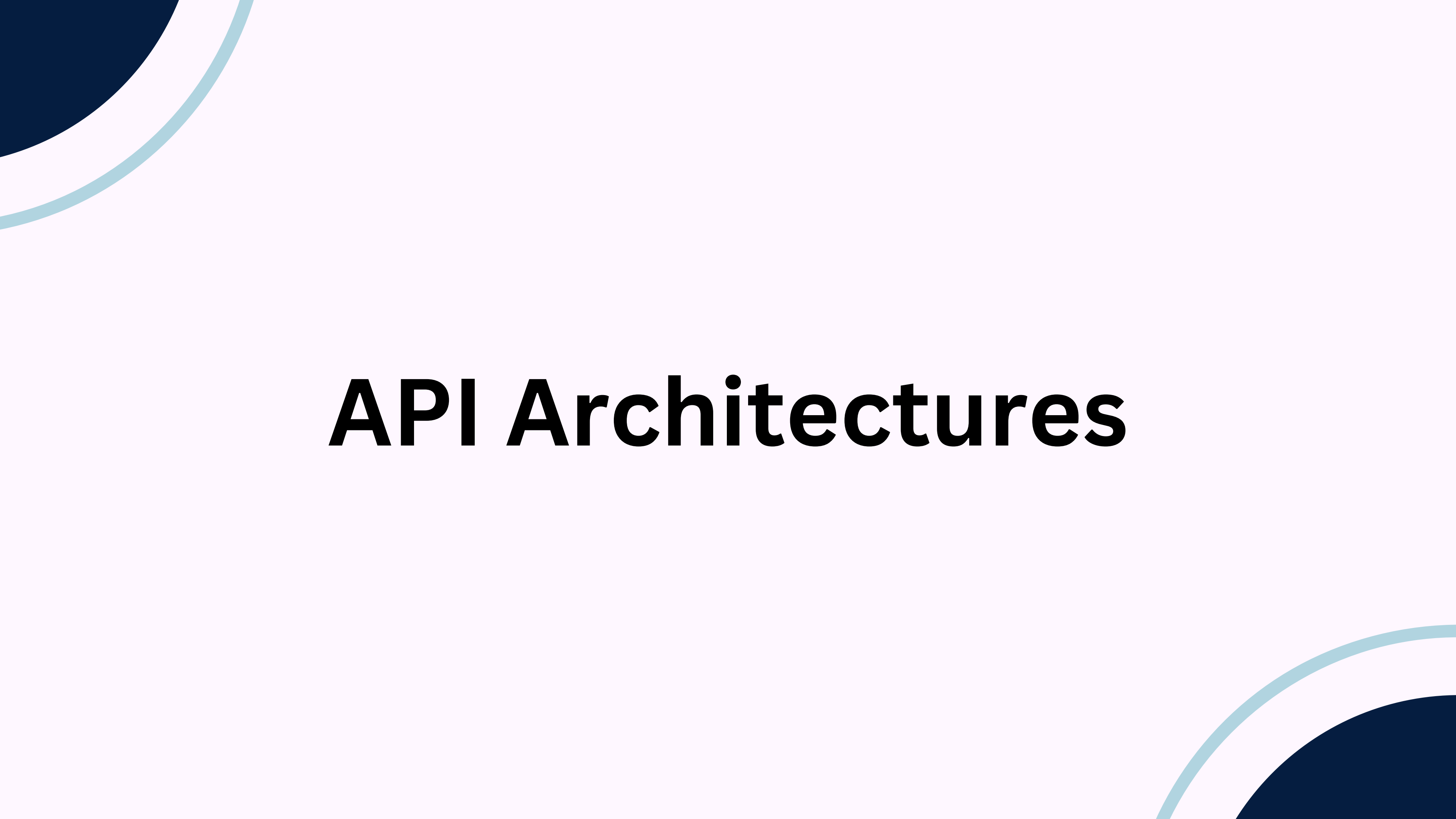
- APIs that facilitate communication between software and hardware devices.
 - Example: A Bluetooth API that allows an application to communicate with Bluetooth-enabled devices, like speakers or fitness trackers.
- 

How Web APIs Fit in Web Application Architecture

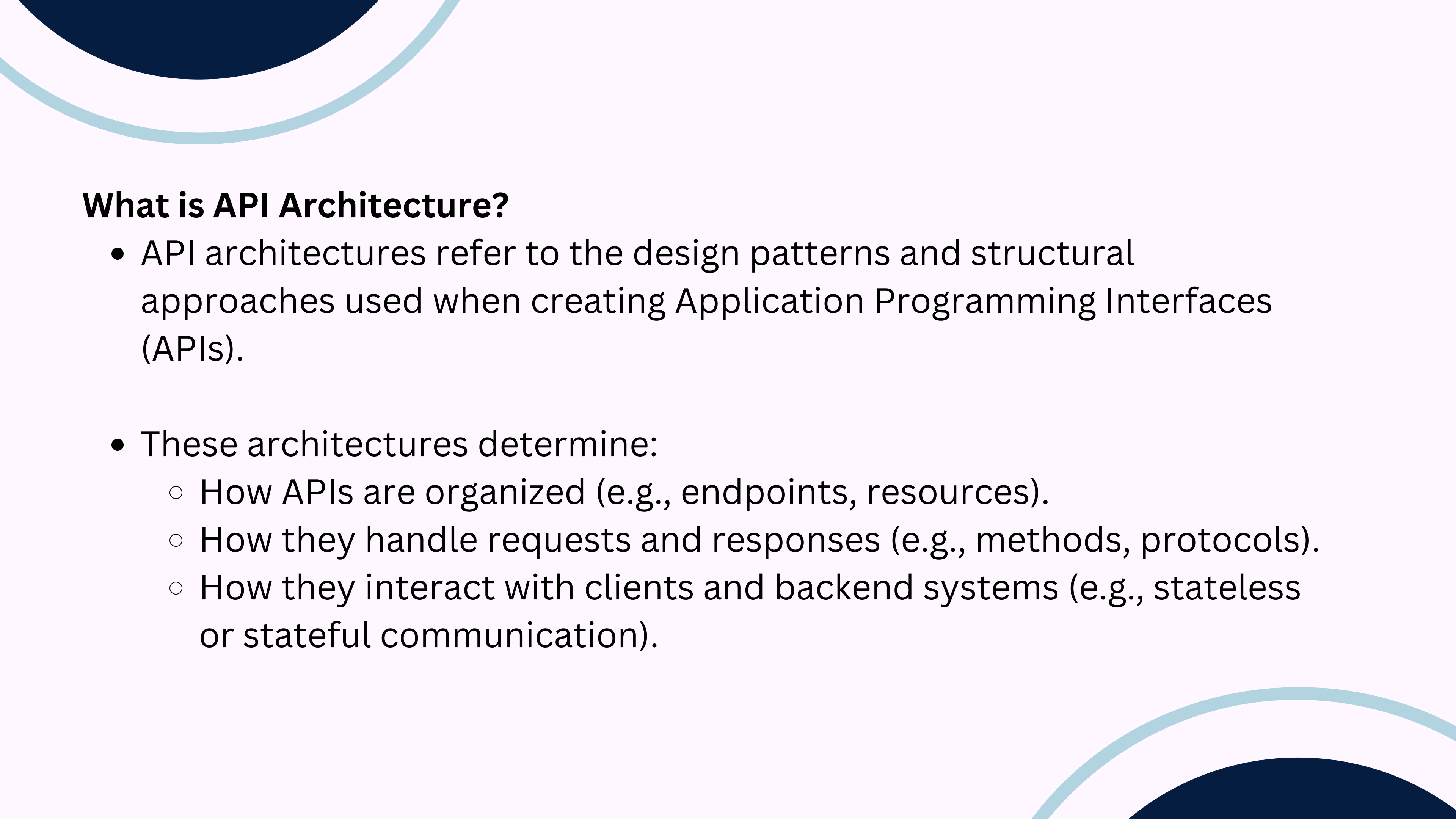


Common Use Cases for Web APIs

- Data Retrieval: Fetching data such as weather forecasts or user profiles.
- Data Submission: Submitting forms or user-generated content (e.g., comments, posts).
- Integration: Connecting to third-party services like payment processors or social media.
- Microservices: Enabling different parts of an application to communicate via APIs.



API Architectures



What is API Architecture?

- API architectures refer to the design patterns and structural approaches used when creating Application Programming Interfaces (APIs).
- These architectures determine:
 - How APIs are organized (e.g., endpoints, resources).
 - How they handle requests and responses (e.g., methods, protocols).
 - How they interact with clients and backend systems (e.g., stateless or stateful communication).

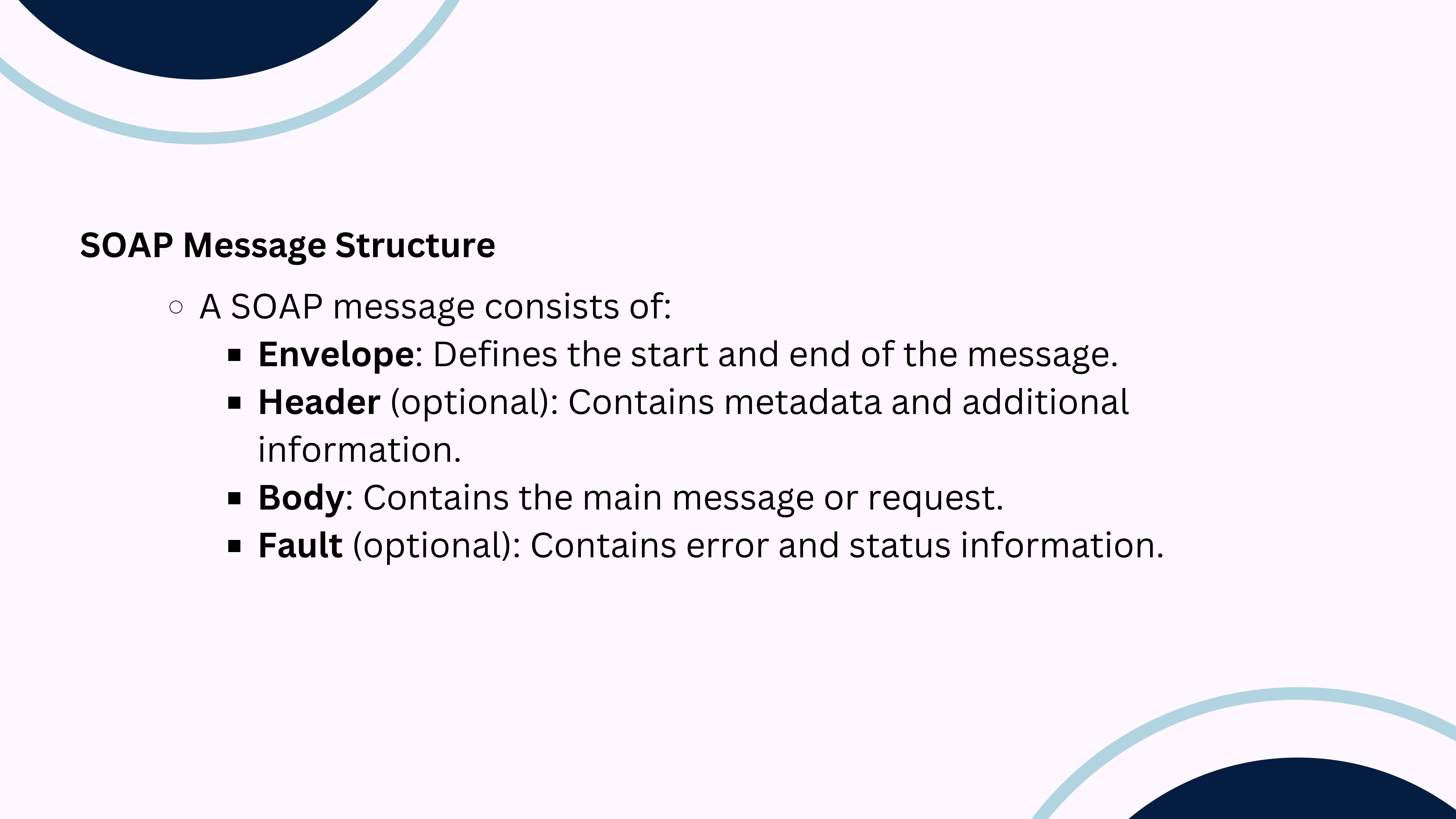
SOAP (Simple Object Access Protocol)

- SOAP is a **protocol** used for exchanging structured information in the implementation of web services.
- Developed by Microsoft in the late 1990s, it is older and more rigid compared to REST and GraphQL.
- *Uses XML for message format*: Ensures strict standards but is verbose.
- Protocol-based: Follows strict rules for communication, like security and transaction handling.
- **Highly Secure**: SOAP supports various security protocols (e.g., WS-Security), making it suitable for financial or critical systems.
- *Requires **more bandwidth***: Due to its heavy XML format, it can be slower and consume more resources.
- **Transport Flexibility**: Can operate over various protocols, such as HTTP, SMTP, or TCP, allowing for diverse communication methods.

- 
- Uses HTTP POST for all requests: SOAP requests are always sent using POST when over HTTP, as they require the **payload** to be sent in the request body.

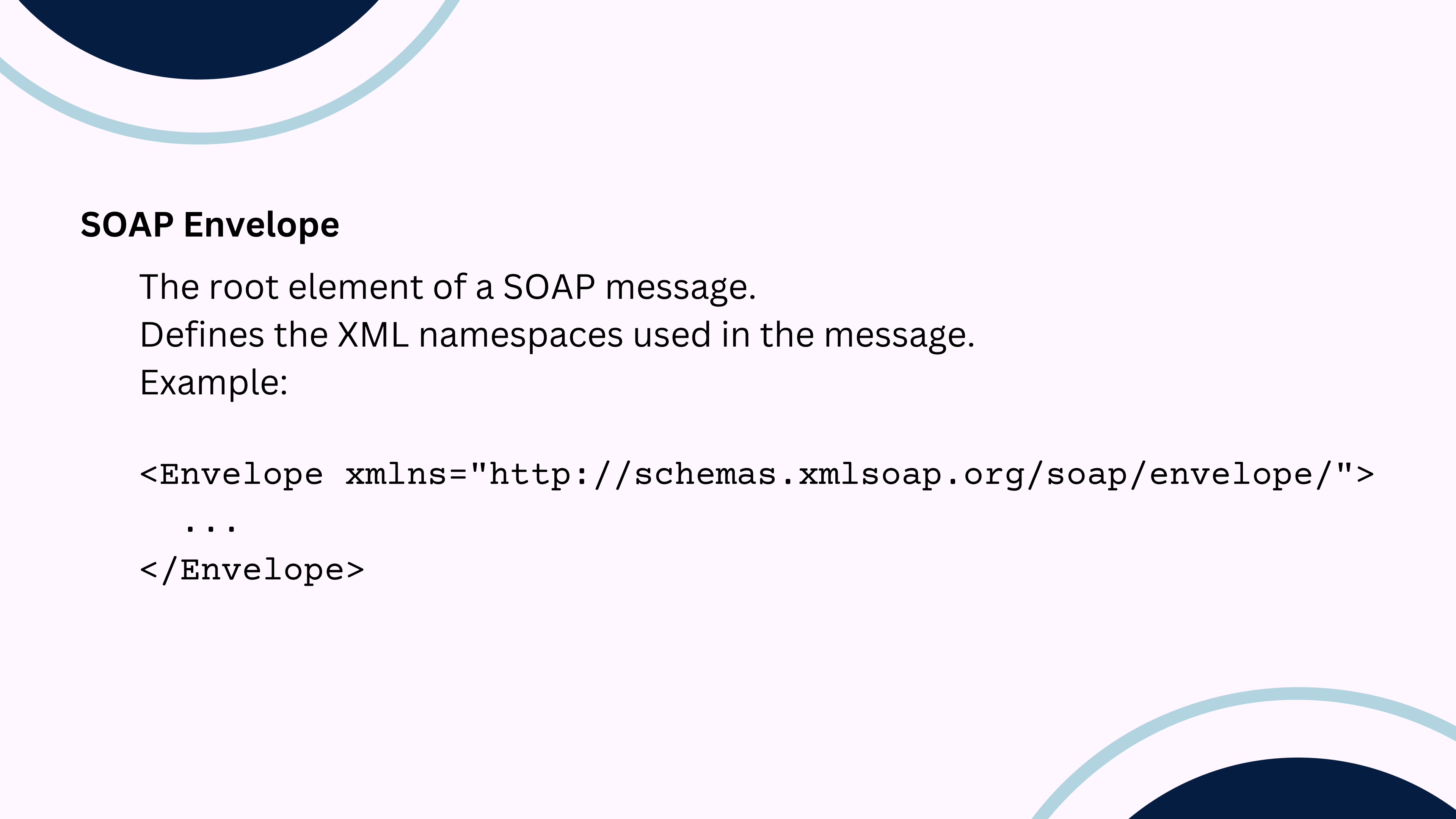
Common Use Cases:

- Financial services (banks, credit card companies).
 - Telecommunications services (network communication).
- 



SOAP Message Structure

- A SOAP message consists of:
 - **Envelope**: Defines the start and end of the message.
 - **Header** (optional): Contains metadata and additional information.
 - **Body**: Contains the main message or request.
 - **Fault** (optional): Contains error and status information.



SOAP Envelope

The root element of a SOAP message.
Defines the XML namespaces used in the message.

Example:

```
<Envelope xmlns="http://schemas.xmlsoap.org/soap/envelope/">  
  ...  
</Envelope>
```

SOAP Header

- An optional part that contains metadata such as authentication or transaction details.
- Can include multiple header entries.
- Example:

```
<Header>
```

```
<Authentication>
```

```
...
```

```
</Authentication>
```

```
</Header>
```



SOAP Body

- Contains the actual message or request information.
- It is mandatory and may include:
 - Request details for a web service.
 - Response data from a service.
- Example:

```
<Body>
```

```
  <GetWeather>
```

```
    <City>London</City>
```

```
  </GetWeather>
```

```
</Body>
```



SOAP Fault

- An optional element used for error handling.
- Provides information about errors that occur during message processing.
- Example:

```
<Fault>  
  <faultcode>soapenv:Client</faultcode>  
  <faultstring>Invalid Request</faultstring>  
</Fault>
```

src: <https://www.postman.com/cs-demo/public-soap-apis/>

```
POST /websamples.countryinfo/CountryInfoService.wso HTTP/1.1
```

```
Host: webservices.oorsprong.org
```

```
Content-Type: text/xml; charset=utf-8
```

```
Content-Length: length
```

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:con="http://www.oorsprong.org/websamples.countryinfo">
  <soapenv:Header/>
  <soapenv:Body>
    <con:CapitalCity>
      <con:sCountryISOCode>US</con:sCountryISOCode>
    </con:CapitalCity>
  </soapenv:Body>
</soapenv:Envelope>
```


HTTP/1.1 200 OK

Content-Type: text/xml; charset=utf-8

Content-Length: length

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                    xmlns:con="http://www.oorsprong.org/websamples.countryinfo">
  <soapenv:Header/>
  <soapenv:Body>
    <con:CapitalCityResponse>
      <con:CapitalCityResult>Washington</con:CapitalCityResult>
    </con:CapitalCityResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

REST (Representational State Transfer)

- REST is an architectural style, not a strict protocol, used for designing networked applications.
- It relies on a stateless, client-server model, where data is typically exchanged in formats like JSON or XML.
- Uses standard HTTP methods like GET, POST, PUT, DELETE for CRUD operations (Create, Read, Update, Delete).
- Lightweight and scalable: REST's simplicity allows for faster performance and lower resource consumption compared to SOAP.
- Data flexibility: Supports multiple formats, though JSON is most commonly used.
- Stateless communication: Each request from client to server must contain all the information needed to understand and process it, without relying on stored context.

```
POST /api/2.2/auth/signin HTTP/1.1
```

```
HOST: my-server
```

```
Content-Type: application/json
```

```
Accept: application/json
```

```
{  
  "credentials": {  
    "name": "administrator",  
    "password": "passw0rd",  
    "site": {  
      "contentUrl": ""  
    }  
  }  
}
```

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "status": "success",
  "message": "User signed in successfully.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "administrator",
    "role": "admin",
    "permissions": ["read", "write", "delete"]
  }
}
```

GraphQL

- GraphQL is a query language developed by Facebook for APIs, giving clients the power to ask for exactly the data they need, and nothing more.
- It's more flexible than REST, allowing multiple resources to be fetched in a single request.

Key Points:

- Client-driven queries: Clients can request exactly the data they need, reducing over-fetching and under-fetching of data.
- Single endpoint: Unlike REST, where you often have multiple endpoints, GraphQL APIs are served via a single endpoint.
- Self-documenting: GraphQL's schema defines the structure of the data available, making it easier for developers to understand and work with the API.
- Can be more complex: Requires upfront effort in defining schemas and managing complex queries.

Request

POST /graphql HTTP/1.1

Host: api.example.com

Content-Type: application/json

Authorization: Bearer YOUR_ACCESS_TOKEN

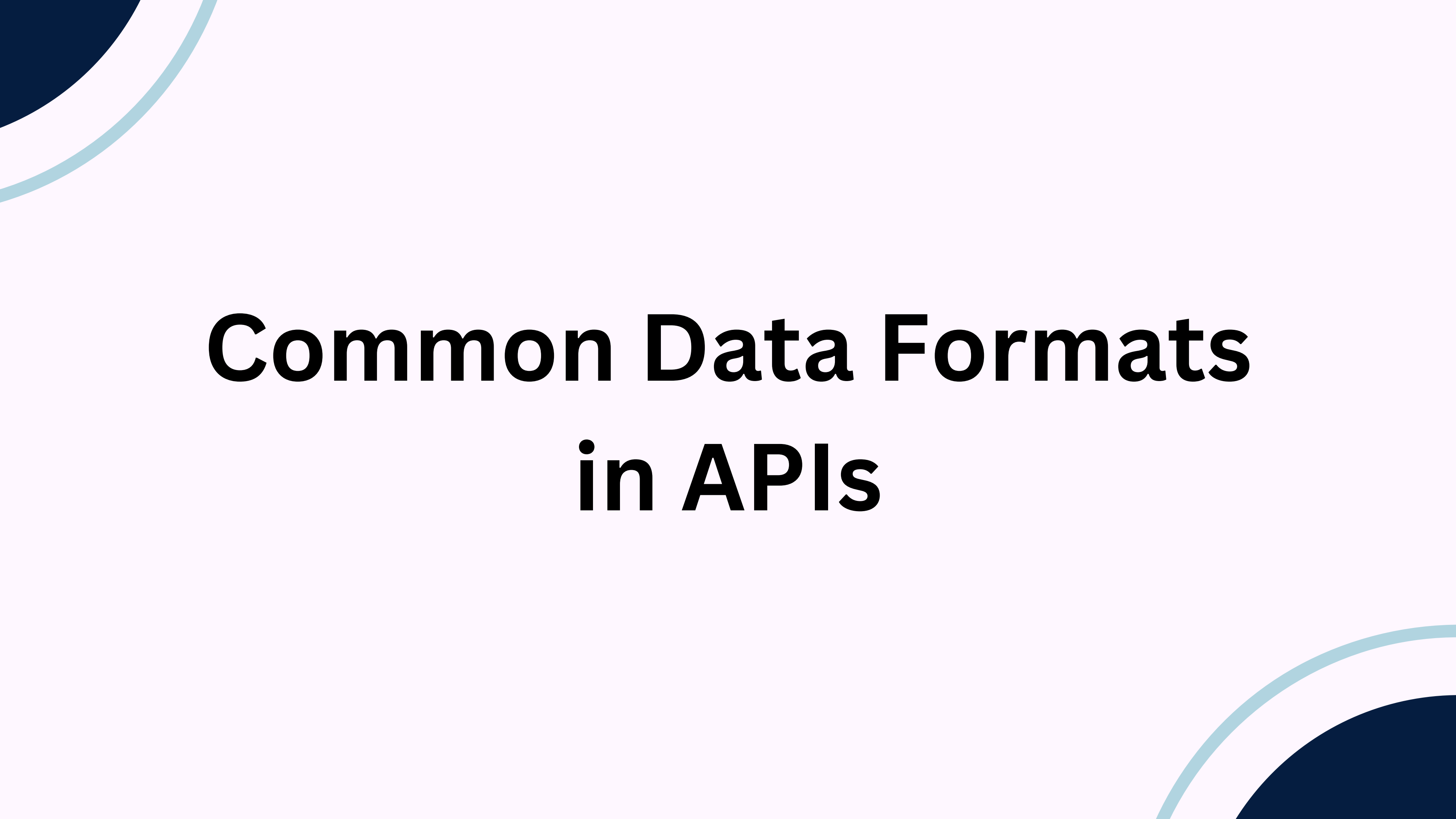
```
{  
  "query": "query GetUserDetails($userId: ID!) { user(id: $userId) { id name email age } }",  
  "variables": {  
    "userId": "12345"  
  }  
}
```

HTTP/1.1 200 OK

Content-Type: application/json

Response

```
{
  "data": {
    "user": {
      "id": "12345",
      "name": "John Doe",
      "email": "john.doe@example.com",
      "age": 30
    }
  },
  "errors": null
}
```



Common Data Formats in APIs

Overview:

- APIs must communicate data effectively between clients and servers. The format of this data is crucial for ensuring successful interactions.
- Data formats define how information is structured, transmitted, and interpreted in API requests and responses.

Importance:

- Interoperability: Different systems can understand and work with the same data, enhancing compatibility.
- Efficiency: The right format can improve data transmission speed and reduce bandwidth usage.
- Readability: Human-readable formats facilitate debugging and manual data manipulation.

JSON (JavaScript Object Notation)

Overview:

- JSON is a lightweight data interchange format that is easy for humans to read and write and easy for machines to parse and generate.
- It is often used in web APIs for exchanging data between clients and servers.

Characteristics:

- Syntax: Consists of key-value pairs enclosed in curly braces {}.
- Data Types: Supports strings, numbers, arrays, booleans, and nested objects.
- Natively Supported in JavaScript: JSON can be easily converted to and from JavaScript objects.

```
{  
  "name": "Alice",  
  "age": 25,  
  "isStudent": false,  
  "courses": ["Math", "Science"],  
  "address": {  
    "street": "123 Main St",  
    "city": "Anytown"  
  }  
}
```

Use Cases:

- **REST APIs:** Commonly used for data exchange in RESTful web services.
- **Web Applications:** Frequently used in AJAX* calls for dynamic content loading.

Benefits:

- **Simplicity:** Easy to read and write, making it developer-friendly.
- **Efficiency:** Generally smaller in size compared to XML, leading to faster transmission.
- **Wide Adoption:** Supported by most programming languages and frameworks.

*AJAX is a technique that enables web applications to send and receive data asynchronously without reloading the entire page. This allows for dynamic updates and improved user experiences, making it an essential tool in modern web development.

JavaScript Objects

- Data Structure: JavaScript objects are in-memory data structures used in JavaScript to store keyed collections of various data and more complex entities.
- Syntax: In JavaScript, object syntax allows for key definitions without quotes (unless they are reserved words or contain spaces), and values can be more diverse, including functions and other objects.
- Use: JavaScript objects are used to represent real-world entities and manage application state, behavior, and logic within JavaScript applications.

Key Differences

1. **Quotation Marks:** JSON keys must be double-quoted, while JavaScript object keys can be unquoted if they are valid identifiers.
2. **Data Types:** JavaScript objects can hold functions and other complex types, while JSON can only hold data types like strings, numbers, arrays, booleans, and objects.
3. **Serialization:** JSON is a string representation that can be transmitted over networks, while JavaScript objects exist in memory and can be manipulated directly in code.

Summary

- JSON is a lightweight format for data interchange, while JavaScript objects are versatile data structures used in JavaScript programming.

```
const user = {  
  name: "Alice",  
  age: 25,  
  isStudent: false,  
  courses: ["Math", "Science"],  
  address: {  
    street: "123 Main St",  
    city: "Anytown"  
  },  
  // Function to add a new course  
  addCourse: function(course) {  
    this.courses.push(course);  
  }  
};
```

XML (eXtensible Markup Language)

Overview:

- XML is a way to store and transport data that uses tags to define elements, making it easy to read for both humans and machines.

Characteristics:

- Descriptive: Uses tags (like <name> and <age>) to clearly label data.
- Flexible Structure: Allows for custom tags to fit different needs.
- Hierarchical: Supports nested elements, enabling complex data organization.

Use Cases:

- Often used in web services and APIs to exchange data, especially in business applications.
- Commonly used for configuration files due to its structured format.

Benefits:

- Customizable: You can create your own tags, making it suitable for various purposes.
- Cross-platform: Works on different systems, allowing easy data sharing.

```
<user>
  <name>Alice</name>
  <age>25</age>
  <isStudent>false</isStudent>
  <courses>
    <course>Math</course>
    <course>Science</course>
  </courses>
  <address>
    <street>123 Main St</street>
    <city>Anytown</city>
  </address>
</user>
```

JSON vs. XML: A Comparison

Feature	JSON	XML
Syntax	Key-value pairs enclosed in curly braces and square brackets	Tags defining elements and attributes
Readability	Generally more human-readable	Can be less readable for complex structures
Data Structure	More flexible with support for arrays and objects	More hierarchical, suited for document-like structures
Performance	Typically faster to parse and transmit	Can be slower, especially for large or complex documents
Common Use Cases	Web development, APIs, data interchange	Configuration files, data interchange, structured documents

APIs Authentication and Authorization

Why We Need Them:

- In today's digital landscape, applications handle sensitive information, such as personal and financial data.
- Without proper security measures, unauthorized users can gain access to this data, leading to potential data breaches, identity theft, and loss of user trust.

What Are They?

- *Authentication*: The process of verifying the identity of a user or system trying to access an API. It ensures that the entity is who they claim to be.
- *Authorization*: The process of determining what an authenticated user is allowed to do. It controls access to resources based on permissions and roles.

1. Authentication (Who Are You?):

- Acts like a security guard checking IDs.
- Users provide credentials (e.g., username and password) to verify their identity.
- Example: Logging into a social media account with an email and password.

2. Authorization (What Can You Do?):

- Determines what authenticated users can access or modify.
- Checks user permissions based on roles.
- Example: A regular user can view content, while an admin can create or delete content.

Importance:

- Security: Protects sensitive information and ensures only legitimate users have access.
- User Trust: Strong authentication and authorization practices build confidence in the application's security measures.

Authentication Methods

Basic Authentication:

- A simple authentication scheme built into the **HTTP** protocol.
- The user's credentials (username and password) are concatenated with a colon and then encoded in Base64.

How It Works:

- The client sends an HTTP request with an Authorization header, containing the word **Basic** followed by the Base64-encoded credentials.
- The server decodes the credentials, verifies them, and then either allows or denies access.

Use Case:

- Often used for APIs that don't require strong security, such as internal applications or quick setups where a simpler approach is sufficient.

Cons:

- Not Secure on Its Own: Base64 is not encryption—it's merely encoding. Anyone with access to the request can decode the credentials if HTTPS is not used.

Example:

- Sending a request with an Authorization header:

```
GET /secure-endpoint HTTP/1.1  
Host: api.example.com  
Authorization: Basic dXNlcjpwYXNz
```

- In this example, dXNlcjpwYXNz is the Base64-encoded string of user:pass.

API Keys

- Description:
 - API Keys are unique identifiers used to authenticate requests from clients (e.g., apps or websites) to the API.
 - They are often passed in the *request headers*, *query parameters*, or as part of the request body.
 - The key is usually a long, randomly generated string that identifies the client and grants it access to specific API functionalities.
- How It Works:
 - When a client (e.g., a web app) wants to access the API, it includes its API Key in the request.
 - The API server checks the key's validity, and if it's valid, it processes the request.

- Pros:
 - Easy to Implement: Straightforward to set up, making it a quick solution for basic API access control.
- Cons:
 - Not Highly Secure: API Keys can be exposed in network traffic if HTTPS is not used, and they can be shared or leaked accidentally.

Bearer Authentication

- A common token-based authentication scheme where a Bearer Token is used to authenticate API requests.
- The token is usually included in the Authorization header of HTTP requests, like a password that grants access to the API.
- The term "Bearer" signifies that anyone who possesses the token can use it to access the protected resources (like a bearer of a ticket).

How It Works:

- After a user successfully logs in, the server issues a Bearer Token, usually in the form of a **JWT** (JSON Web Token).
- The client includes this token in the Authorization header for each subsequent request to access secure endpoints.
- The server verifies the token before granting access to the requested resources.

Example:

```
GET /user/profile HTTP/1.1  
Host: api.example.com  
Authorization: Bearer abc123xyz-token
```

- The Authorization header must start with the word "Bearer" followed by a space and then the token

Pros:

- Easy to Use: Clients just need to include the token in the request header, simplifying the process.
- Stateless: The server does not need to store session information; all necessary data is embedded within the token.

Cons:

- Less Secure If Compromised: Anyone who gets hold of the Bearer Token can access the API as long as the token is valid.
- Token Management: Managing expiration and renewal of tokens is necessary to maintain security.

What is JWT (JSON Web Token)?

What is JWT?

- Definition:
 - JSON Web Token (JWT, pronounced “jot”) is a compact, URL-safe means of representing claims to be transferred between two parties.
- Use Case:
 - Commonly used for authentication and information exchange in web applications.

Structure of a JWT

- JWTs are made up of three parts, separated by dots (.):
 - Header: { "alg": "HS256", "typ": "JWT" }
 - Payload: { "sub": "1234567890", "name": "John Doe", "iat": 1516239022 }
 - Signature: A hash created using the header, payload, and a secret key.
- Visual Example:
 - Header.Payload.Signature
 - Example:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODk1IiwiaWF0IjE0NjQ5OTIyMj0.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

How JWT Works

- 1. Client Login: User logs in with credentials.
- 2. Server Generates JWT: Server creates and signs a JWT using a secret key.
- 3. Client Stores JWT: Typically stored in localStorage or as a cookie.
- 4. Client Sends JWT in Requests: JWT is sent in the Authorization header as a bearer token: Authorization: Bearer <token>.
- 5. Server Verifies JWT: Server checks the signature to verify the token's authenticity and extracts user information.

OAuth 2.0

Cookie Authentication

Cookie authentication is a method used to identify and authenticate users through HTTP cookies stored in the user's browser. When a user logs in, the server sends a cookie that contains a session identifier, allowing the user to remain authenticated during their session.

How It Works:

1. The user submits their credentials (username and password) to the server.
2. Upon successful login, the server creates a session and sends a Set-Cookie header with a session ID to the user's browser.
3. For subsequent requests, the browser automatically includes the cookie, allowing the server to recognize the user without requiring re-authentication.

Authorization Methods

Remember:

Authorization is the process of determining what an authenticated user is allowed to do within an application or API. It defines the permissions and access rights for users after their identity has been verified through authentication.

2. Importance of Authorization

- *Resource Protection*: Ensures that sensitive data and actions are only accessible to users with the proper permissions.
- *User Experience*: Allows for customized experiences based on user roles and permissions.

Authorization Models

- Role-Based Access Control (RBAC):
 - Users are assigned roles that define their access levels (e.g., admin, user, guest).
- Attribute-Based Access Control (ABAC):
 - Access is granted based on attributes (e.g., user attributes, resource attributes, environmental conditions).
- Access Control Lists (ACL):
 - Specific permissions are defined for each user or group on resources.

API Documentation

- What is API Documentation?
 - A guide that explains how to effectively use and integrate with an API.
 - It typically includes details about API endpoints, request and response formats, error codes, and example requests.

Importance of Well-Written API Documentation

1. Clarity & Understanding

- Ease of Use: Enables quick understanding of endpoints, requests/responses, and authentication.
- Detailed Descriptions: Clear input parameters, outputs, and error codes simplify integration.

2. Reduces Miscommunication

- Standardization: A single source of truth ensures consistency.
- Minimizes Errors: Clear examples prevent misuse and reduce bugs.

3. Saves Time & Resources

- Reduces Support Requests: Users rely less on support with thorough documentation.
- Speeds Up Development: Faster understanding leads to quicker integration.

4. Improves Onboarding

- Eases Learning: New developers can get up to speed faster.
- Promotes Adoption: Comprehensive docs attract more users to public APIs.

Key Elements of Good API Documentation

1. Clear Endpoint Descriptions

- Include URL paths, HTTP methods (GET, POST, etc.), and a brief description of each endpoint.

2. Request and Response Examples

- Provide sample requests with parameters and sample responses (e.g., JSON).
- Highlight required headers and query parameters.

3. Authentication Details

- Explain how to authenticate (e.g., API keys, Bearer tokens, OAuth).
- Include example headers for authenticated requests.

4. Error Codes and Messages

- List common error codes like 400, 401, 404, and 500.
- Include descriptions directly in the response definitions to explain the error

6. Interactive Documentation

- Use tools like Swagger/OpenAPI to allow users to try out API calls directly from the documentation.

Introduction to API Documentation Tools

Purpose of API Documentation Tools:

- *Facilitate Understanding:* Help developers and users quickly grasp how to interact with the API.
- *Reduce Communication Overhead:* Minimize the need for back-and-forth discussions by providing clear and comprehensive documentation.

Key Benefits:

- *Consistency:* Maintain a uniform format across documentation, which aids usability.
- *Version Control:* Allow for easier tracking of changes and updates in the API over time.
- *Ease of Use:* Many tools offer user-friendly interfaces that make it easy to create and manage documentation without extensive technical skills.
- *Interactive Features:* Tools often include features for testing API endpoints directly within the documentation, improving developer experience.

Popular API Documentation Tools

- Swagger/OpenAPI
 - Overview: A widely-used framework for defining APIs in a standard, machine-readable format.
 - Key Features:
 - Automatic generation of interactive documentation.
 - Ability to generate client libraries and server stubs.

- Postman
 - Overview: A powerful tool for API development that allows users to build, test, and document APIs.
 - Key Features:
 - Create collections of API requests.
 - Shareable documentation with built-in testing capabilities.

- Redoc
 - Overview: A tool for rendering OpenAPI specifications into beautiful, interactive documentation.
 - Key Features:
 - Customizable themes.
 - Responsive design for different screen sizes.

- GraphQL Playground
 - Overview: An interactive in-browser IDE for exploring GraphQL APIs.
 - Key Features:
 - Real-time API exploration.
 - Automatically generated documentation for GraphQL schemas.

What is OpenAPI

- *Definition of OpenAPI*
 - Standard Specification: OpenAPI is a specification for defining APIs in a standard, machine-readable format, enabling clear communication between API providers and consumers.
 - Formerly Known As: It was previously known as Swagger Specification.
- *Purpose of OpenAPI*
 - Interoperability: Ensures that APIs can be easily understood and integrated across different platforms and languages.
 - Automation: Facilitates automated documentation generation, testing, and client library creation.

Benefits of Using OpenAPI

- Consistency: Provides a standardized way to describe APIs, which helps maintain uniformity across different documentation.
- Rich Ecosystem: Supports a wide range of tools for development, testing, and documentation (e.g., Swagger UI, Postman).
- Versioning and Change Management: Simplifies the process of managing changes and versioning APIs over time.

Structure of OpenAPI Specification

- Basic Components:
 - Info: Metadata about the API (title, version, description).
 - Paths: Defines the available endpoints and their operations.
 - Components: Reusable components (schemas, responses, parameters).

What is Swagger?

- Framework for API Documentation: Swagger is a suite of tools built around the OpenAPI Specification that helps developers design, build, document, and consume RESTful APIs.
- Evolution: Initially started as a standalone tool for documenting APIs, it has evolved to support the OpenAPI standard.

Core Components of Swagger:

- Swagger Editor:
 - Purpose: A web-based editor that allows users to write OpenAPI specifications in YAML/JSON format and see real-time feedback.
- Swagger UI:
 - Purpose: Generates interactive documentation from an OpenAPI specification, enabling users to test API endpoints directly from the browser.
- Swagger Codegen:
 - Purpose: Generates client libraries and server stubs in various programming languages based on the API specification.

Hands-On Example: Documenting an API with Swagger

Introduction to Swagger Editor

- Purpose: A web-based tool for writing and testing OpenAPI specifications in YAML or JSON format.
- Accessibility: Can be accessed directly via a browser (no installation needed).
- Getting Started with Swagger Editor
- Accessing the Editor:
 - URL: Provide the URL for the online Swagger Editor (<https://editor.swagger.io>).

Basic Structure of OpenAPI Specification

- Components of an OpenAPI Spec:
 - *Info Section*: Basic metadata about the API (title, version, description).
 - *Servers Section*: Defines the base URLs where the API is hosted.
 - *Paths Section*: Where endpoints and their operations are defined.
 - *Components Section*: Where reusable parts like schemas are defined.

Note: While the OpenAPI Specification includes additional sections like tags, security, and external documentation, we'll focus on these core components as they form the foundation for building and documenting APIs effectively.

Practical Example: Documenting an API

- Setting Up the Info Section
 - *What is the Info Section?:* The info section in an OpenAPI specification provides metadata about the API, including its title, description, version, and more.
 - *Purpose:* Helps users understand the API at a glance, including its purpose and versioning.

Example Info Section

```
info:  
  title: Product API  
  description: This API allows users to retrieve and manage products.  
  version: 1.0.0  
  contact:  
    name: API Support  
    url: http://www.example.com/support  
    email: support@example.com
```

- Key Components Explained:
 - *Title*: A human-readable name for the API that appears in the documentation.
 - *Description*: A brief overview of the API's functionality, giving users insight into what it does.
 - *Version*: Indicates the current version of the API, essential for version control and user awareness.
 - *Contact Information*: Provides a way for users to reach out for support or questions regarding the API.

Setting Up the Servers Section

- What is the Servers Section?: Defines the base URLs where the API is hosted, specifying different environments.
- Purpose: Indicates where users can send requests to the API.

Example Servers Section

```
servers:
```

- url: `https://api.example.com/v1`
description: Production server (uses live data)
- url: `https://staging.api.example.com/v1`
description: Staging server (uses test data)\

- Key Components Explained:
 - *URL*: The base URL for the API endpoints, which users will use to make requests.
 - *Description*: Clarifies the function of the server, such as whether it's a production or staging environment.
- Why the Servers Section Matters
 - Endpoint Location: Clearly specifies where users can find the API endpoints.
 - Environment Clarity: Helps users distinguish between different server environments (production vs. staging, for example).

Defining Your First API Endpoint

Creating a Sample Endpoint

- Example Endpoint: Let's define a simple GET /products endpoint.
- Purpose: This endpoint retrieves a list of products from the API.

```
paths:
  /products:
    get:
      summary: Retrieve a list of products
      description: Returns a list of products available in the store.
      responses:
        '200':
          description: A list of products
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Product'
        '204':
          description: No products found
```

Request Body

- Purpose: The request body is used to send data to the server when creating a new product.
- Schema Definition: The schema defines the expected structure of the product data.

- **Endpoints Defined Under Paths:**

- All API endpoints are specified within the paths section of the OpenAPI specification.

- **Key Components Explained:**

- Path: /products
- Method: GET
- Responses:
 - 200: Indicates a successful retrieval of products, with a response body.
 - 204: Indicates that the request was successful, but there are no products to return (no response body).

Adding Request Body and Responses: POST /products

POST Example: Adding a New Product

- Overview: This endpoint allows users to add a new product to the store.

```
paths:
  /products:
    post:
      summary: Add a new product
      description: Creates a new product in the store.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Product'
      responses:
        '201':
          description: Product created successfully
        '400':
          description: Invalid input
```

Request Body

- Purpose: The request body is used to send data to the server when creating a new product.
- Schema Definition: The schema defines the expected structure of the product data.

```
components:
  schemas:
    Product:
      type: object
      properties:
        name:
          type: string
        price:
          type: number
        category:
          type: string
      required:
        - name
        - price
```


- **201 Response:**
 - Description: Indicates the product was created successfully.
- **400 Response:**
 - Description: Indicates there was an issue with the input data (e.g., missing required fields).

Testing APIs with Swagger UI

- Swagger UI provides an interactive interface to test API endpoints straight from the documentation.
- *Purpose:* Great for quickly checking endpoint functionality without needing separate tools.

Steps to Test an API Endpoint

- Step 1: Open the Swagger UI for your API.
- Step 2: Locate the endpoint (e.g., GET /products).
- Step 3: Click "Try it out," enter parameters, and click "Execute."

Benefits of Using Swagger UI for Testing:

- *Ease of Use*: Ideal for quick testing right from the browser.
- *Real-Time Feedback*: Instantly shows responses with status codes, headers, and body content.
- *Interactive Documentation*: Allows users to understand how to use the API with hands-on experience.

Limitations of Swagger UI for Testing:

- *Basic Testing Only: Best for simple endpoint testing, not complex test cases*
- *Missing Test Features: No test cases, assertions, or test reporting*
- *Limited Automation: Can't create automated tests or test scripts - use tools like Postman for that*

API Testing Using Postman

- **What is Postman?**

- A popular API testing tool that simplifies the process of sending requests and analyzing responses.
- Allows for easy collaboration and sharing of APIs.

Installing Postman

1. Download Postman: Visit the Postman website and choose the version for your OS (Windows, macOS, Linux).
2. Install: Follow the installation prompts based on your OS.
3. Create an Account (Optional): Sign up for free to access collaboration features, or use Postman without an account.
4. Launch Postman: Open the app and explore the interface.

Key Features of Postman

- User-Friendly Interface
- Collections for Organizing Requests
- Environment Management for Variables
- Automated Testing Capabilities
- Collaboration Tools for Teamwork
- API Documentation Generation

Creating Your First Request

- Step-by-Step Guide:
 - a. Open Postman and create a new request.
 - b. Choose the HTTP method (GET, POST, etc.).
 - c. Enter the API endpoint URL.
 - d. Add any necessary headers or body data.
 - e. Click "Send" to execute the request.

Writing Tests in Postman

- Introduction to Tests:
 - Postman allows you to write JavaScript tests for your requests.
- Example Test Snippet:

```
pm.test("Response time is less than 200ms", function () {  
    pm.expect(pm.response.responseTime).to.be.below(200);  
});
```

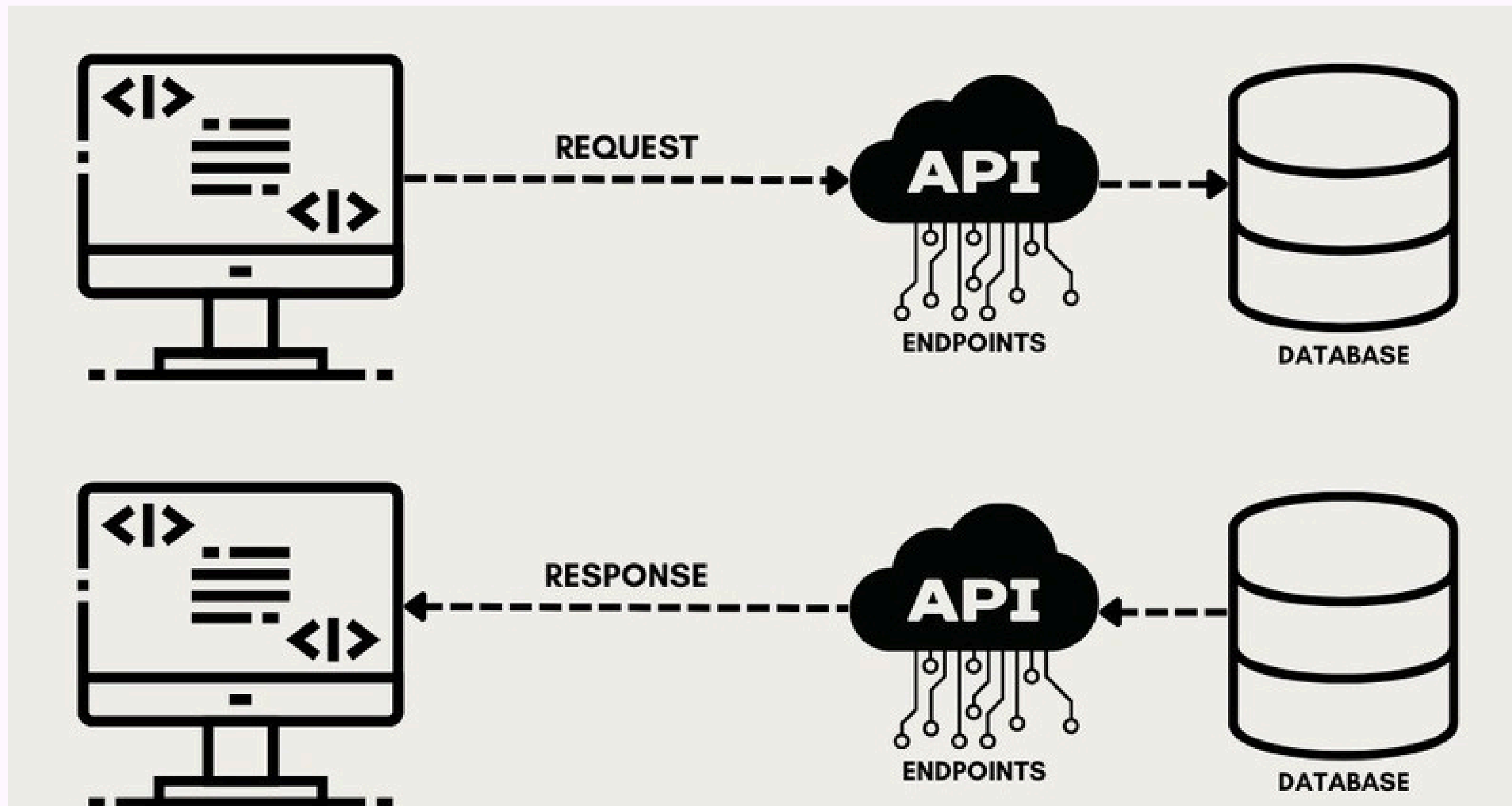

Using Postman Collections

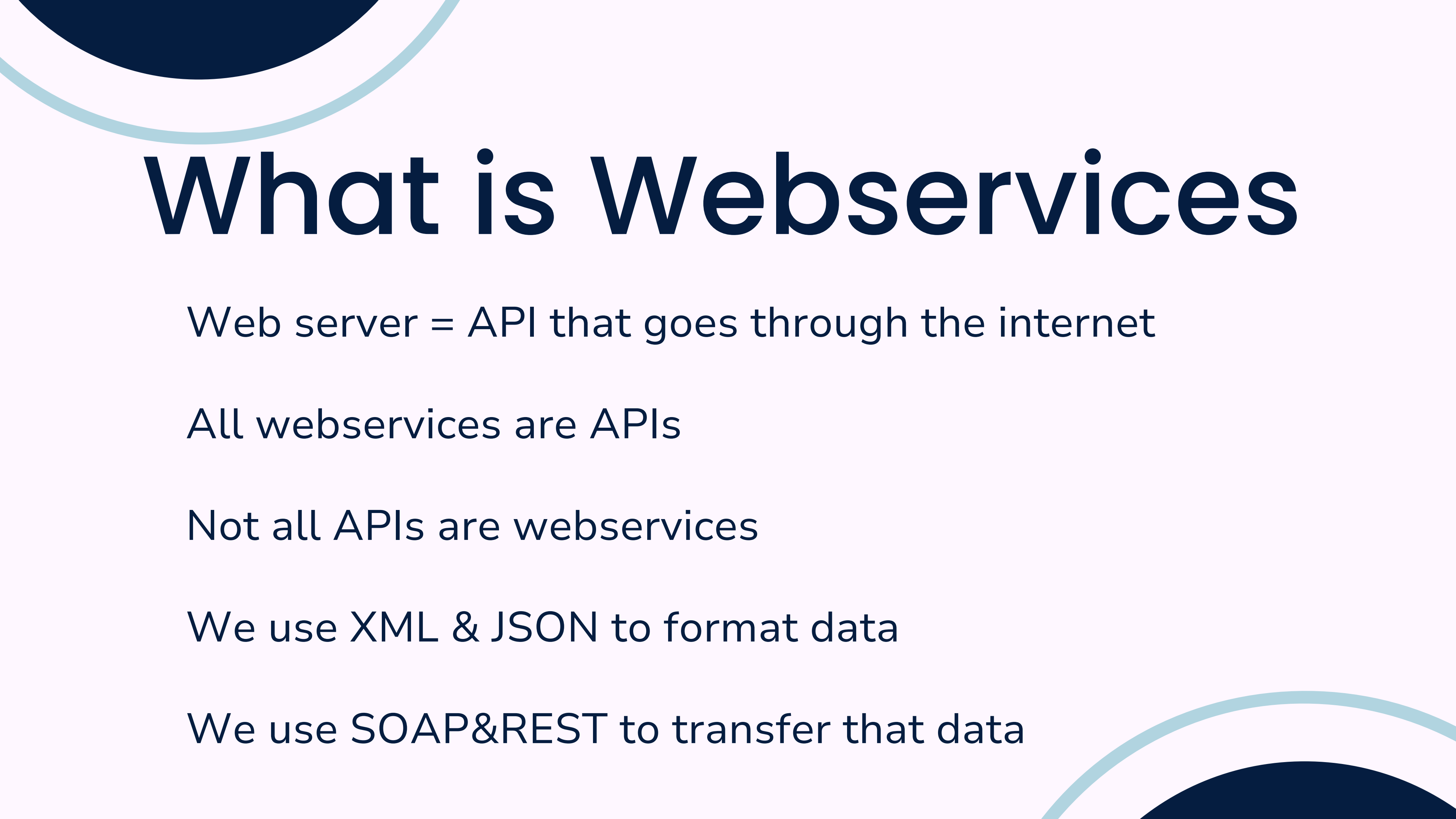
- Creating Collections:
 - Group related requests together.
- Running Collections:
 - Use the Collection Runner to execute multiple requests in sequence.
- Sharing Collections:
 - Share collections with team members or export them for others to use.

Managing Environments

- What Are Environments?
 - Save sets of variables (e.g., API keys, base URLs) to switch easily between different setups (e.g., development, production).
- Why Use Environments?
 - Easily switch between different configurations (e.g., development, staging, production) without modifying requests.
- Using Variables in Requests:
 - Reference variables in URLs or body data using `{{variableName}}`.

What happen in api?





What is Webservices

Web server = API that goes through the internet

All webservices are APIs

Not all APIs are webservices

We use XML & JSON to format data

We use SOAP&REST to transfer that data



API Data

XML & JSON

What is XML?

eXtensible

X

Markup

M

Language

L

XML Example

```
<note>  
  <to>Tarek</to>  
  <from>Mohamed</from>  
  <heading>Reminder</heading>  
  <body>Don't forget your assignment today</body>  
</note>
```

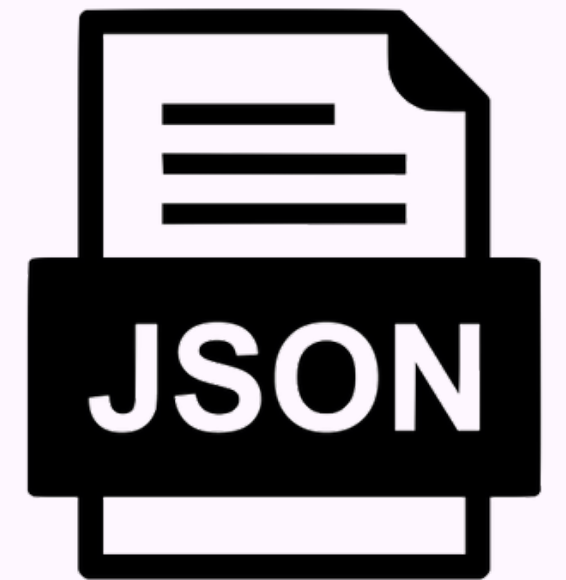

What is JSON

Java**S**cript **O**bject **N**otation “JSON”

It's the part of javascript that hold data

JSON code is smaller than XML

REST APIs use JSON to transfer data



What is JSON

Strings:

Described with double quotes

"Welcome to the API Course"

Lists:

Described with square brackets

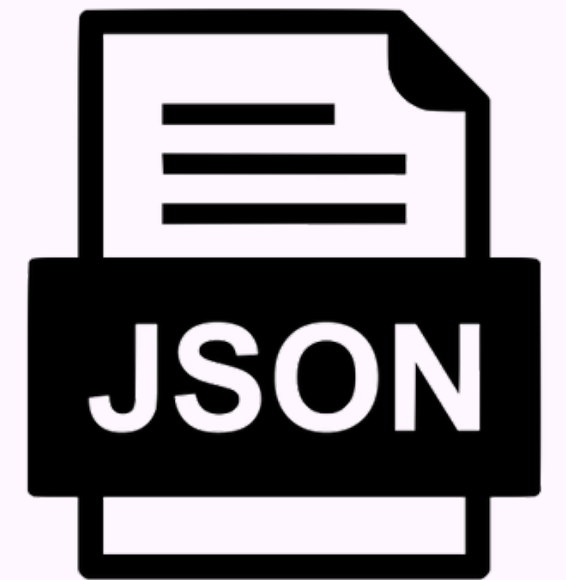
[1, 2, 3]

Objects(collection of key-value pairs):

Described with curly brackets

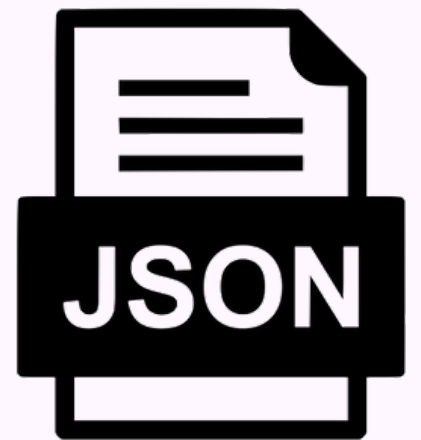
{"key":"value"}

{"username":"tarek"}



JSON Example

```
{  
  "Course One":  
    {"category": "Software Development",  
      "Section": "Software Testing",  
      "Target Audience": ["Students", "Software Engineers"]  
    },  
  "Course Two":  
    {"category": "Software Development",  
      "Section": "Performance Testing",  
      "Target Audience": ["Students", "Software Engineers"]  
    },  
}
```



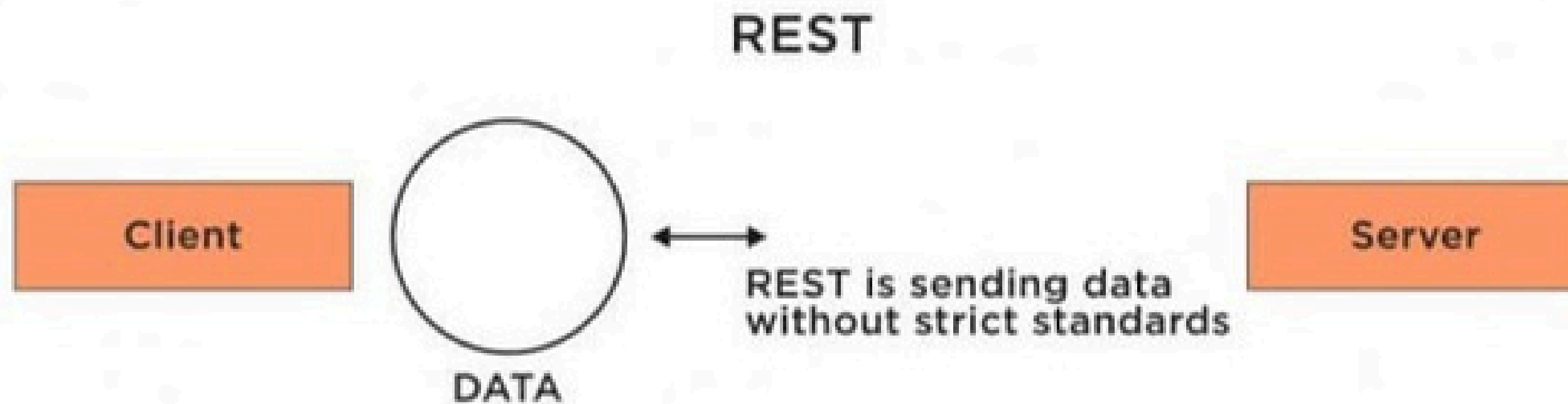
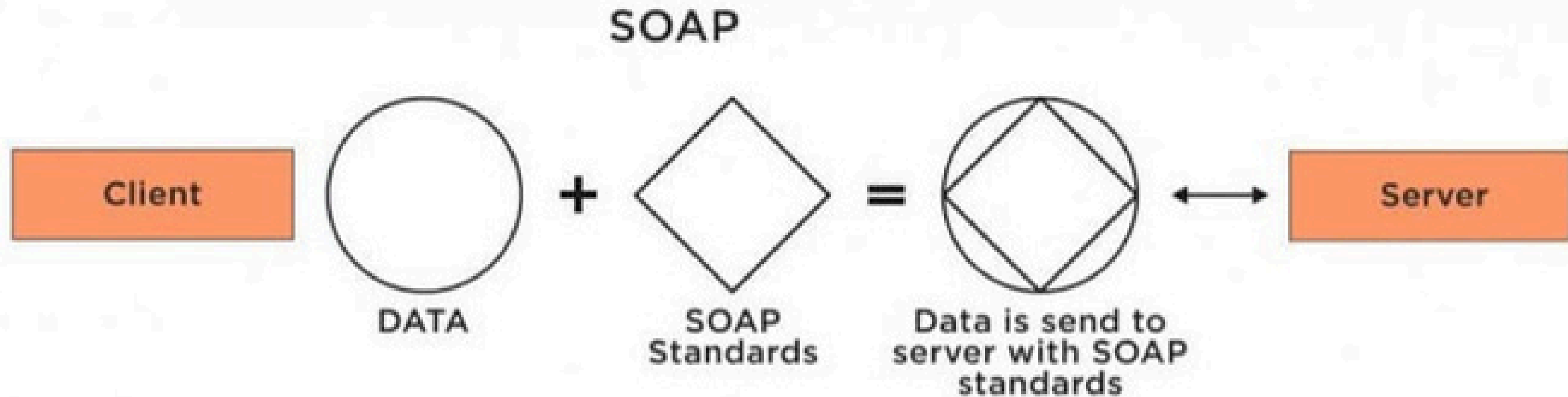
JSON Example

```
{  
  1 "Course One":  
    {  
      2 "category": "Software Development",  
        "Section": "Software Testing",  
        3 "Target Audience": ["Students", "Software Engineers"]  
    },  
  1 "Course Two":  
    {  
      2 "category": "Software Development",  
        "Section": "Performance Testing",  
        3 "Target Audience": ["Students", "Software Engineers"]  
    }  
}
```

```
{ "widget":  
  { "debug": "on",  
    "window": {  
      "title": "Sample Konfabulator Widget",  
      "name": "main_window",  
      "height": 500  
    },  
    "image": {  
      "src": "images/Sun.png",  
      "alignment": "center"  
    },  
    "text": {  
      "data": "Click Here",  
      "size": 36,  
      "style": "bold",  
      "name": "text1",  
      "alignment": "center" }  
  }  
}
```



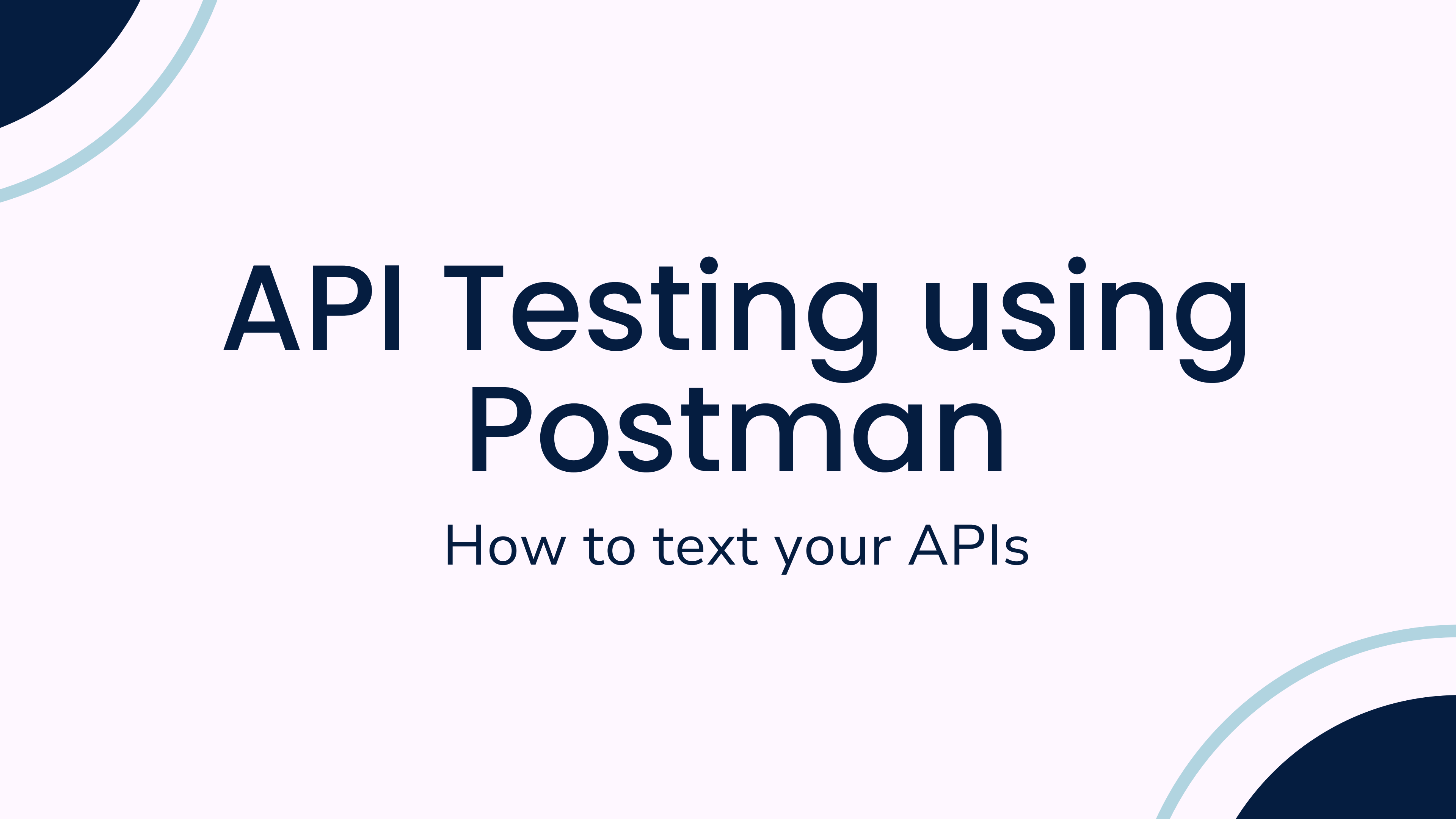
SOAP VS. REST API





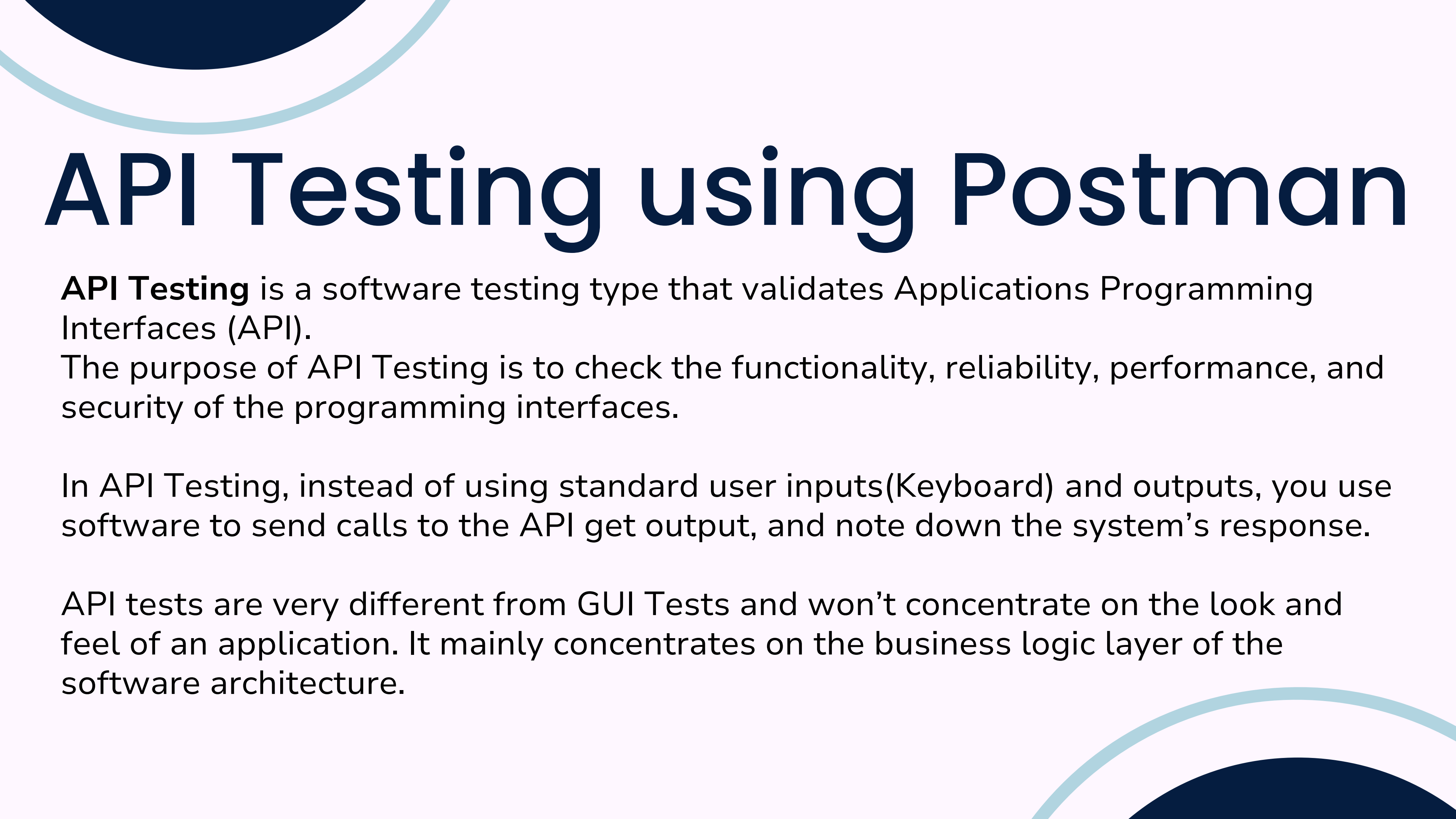
Creating Mock API

Using Mock API & Mocky



API Testing using Postman

How to test your APIs



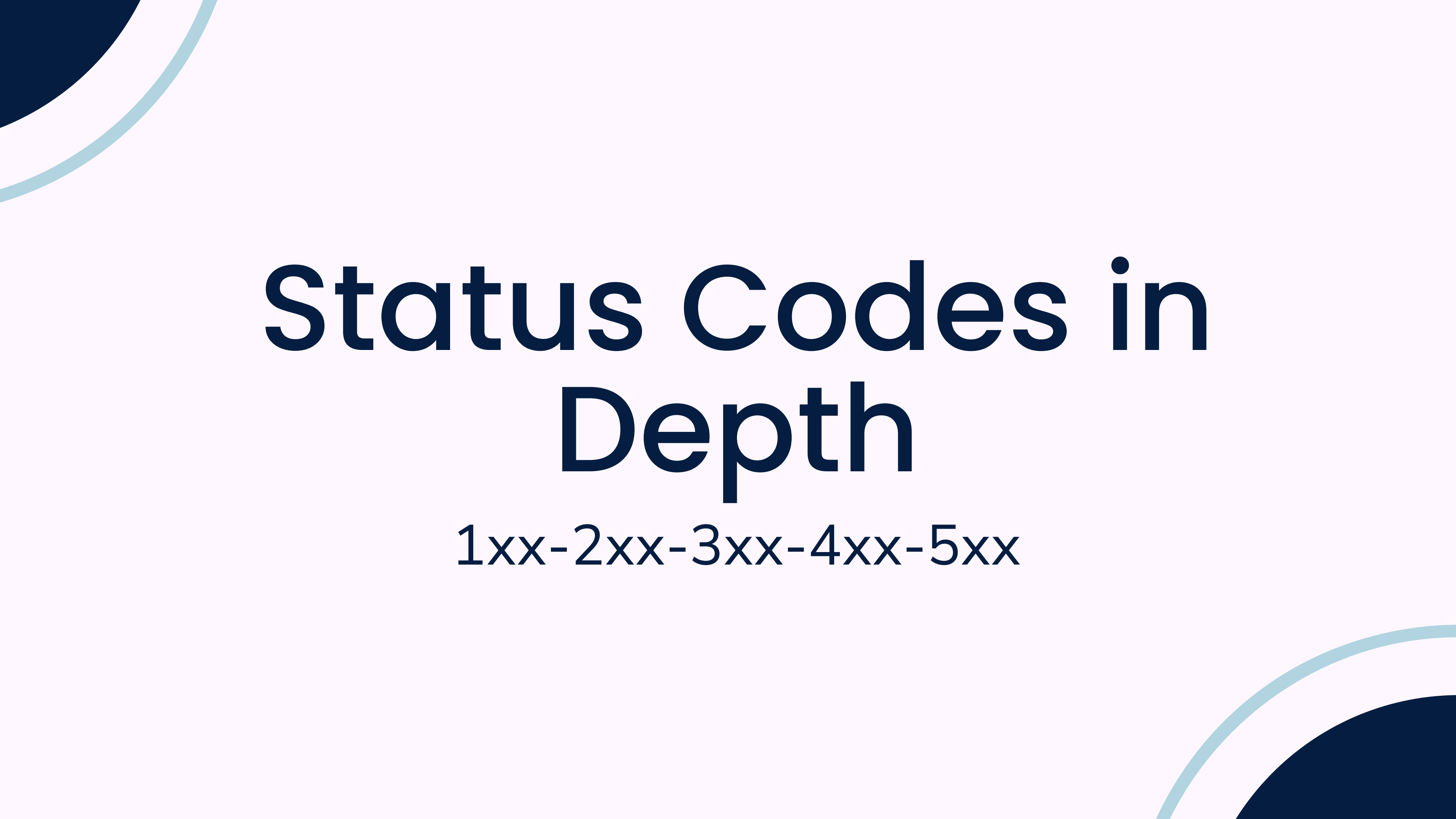
API Testing using Postman

API Testing is a software testing type that validates Applications Programming Interfaces (API).

The purpose of API Testing is to check the functionality, reliability, performance, and security of the programming interfaces.

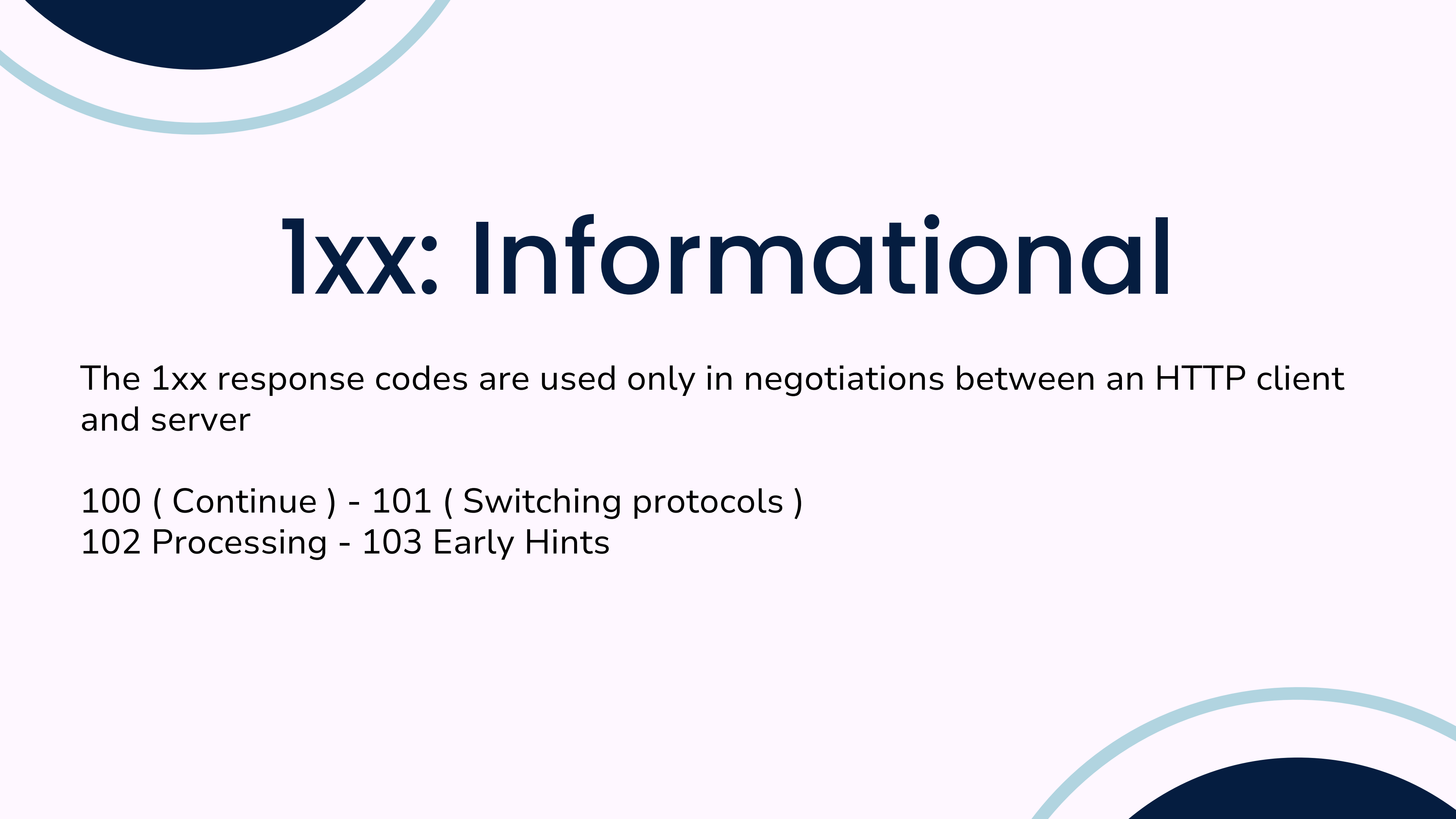
In API Testing, instead of using standard user inputs (Keyboard) and outputs, you use software to send calls to the API get output, and note down the system's response.

API tests are very different from GUI Tests and won't concentrate on the look and feel of an application. It mainly concentrates on the business logic layer of the software architecture.



Status Codes in Depth

1xx-2xx-3xx-4xx-5xx



1xx: Informational

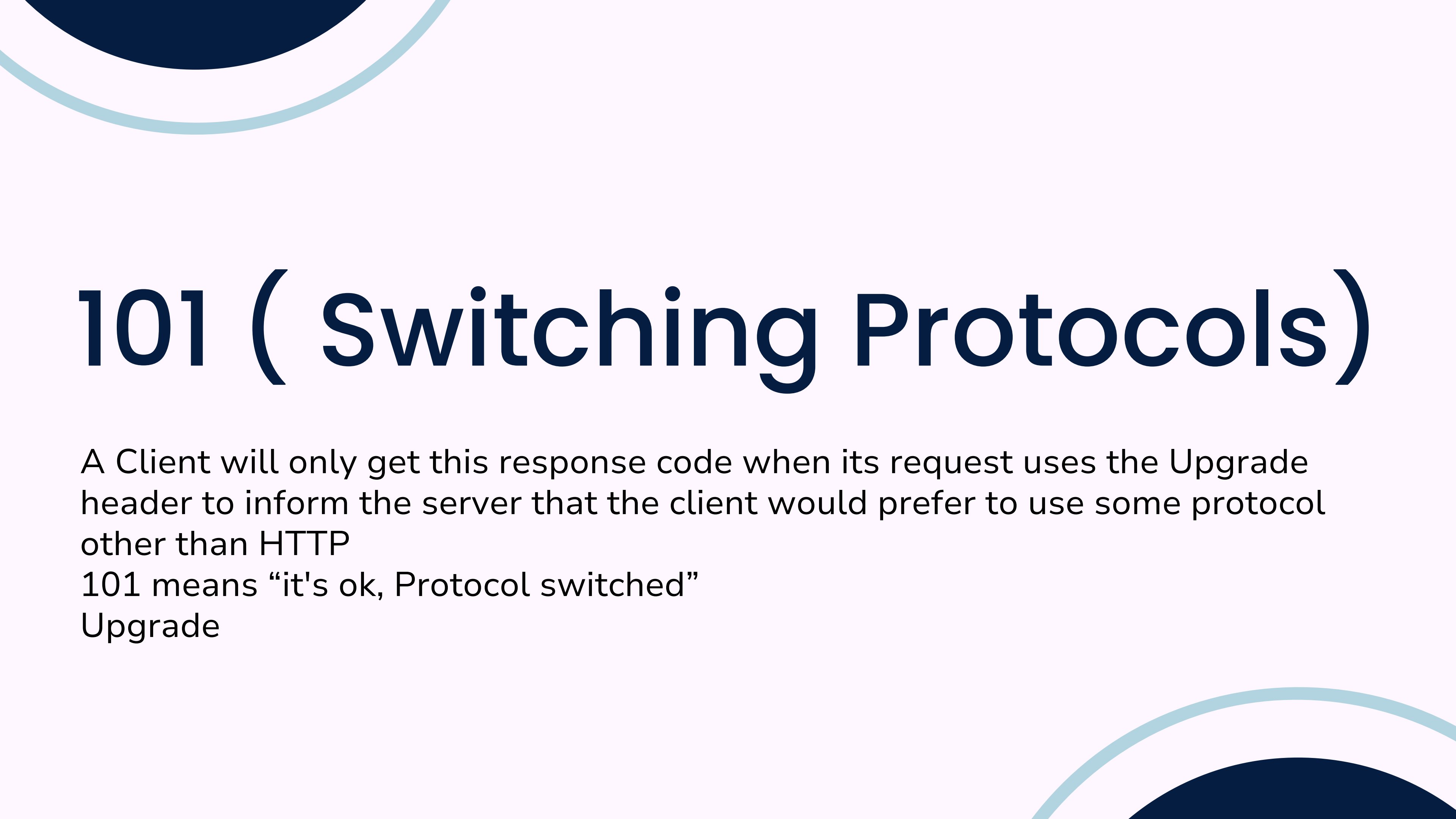
The 1xx response codes are used only in negotiations between an HTTP client and server

100 (Continue) - 101 (Switching protocols)
102 Processing - 103 Early Hints



100 (Continue)

The HTTP 100 Continue informational status response code indicates that everything so far is Ok and the client should continue with the request or ignore it if is already finished



101 (Switching Protocols)

A Client will only get this response code when its request uses the Upgrade header to inform the server that the client would prefer to use some protocol other than HTTP

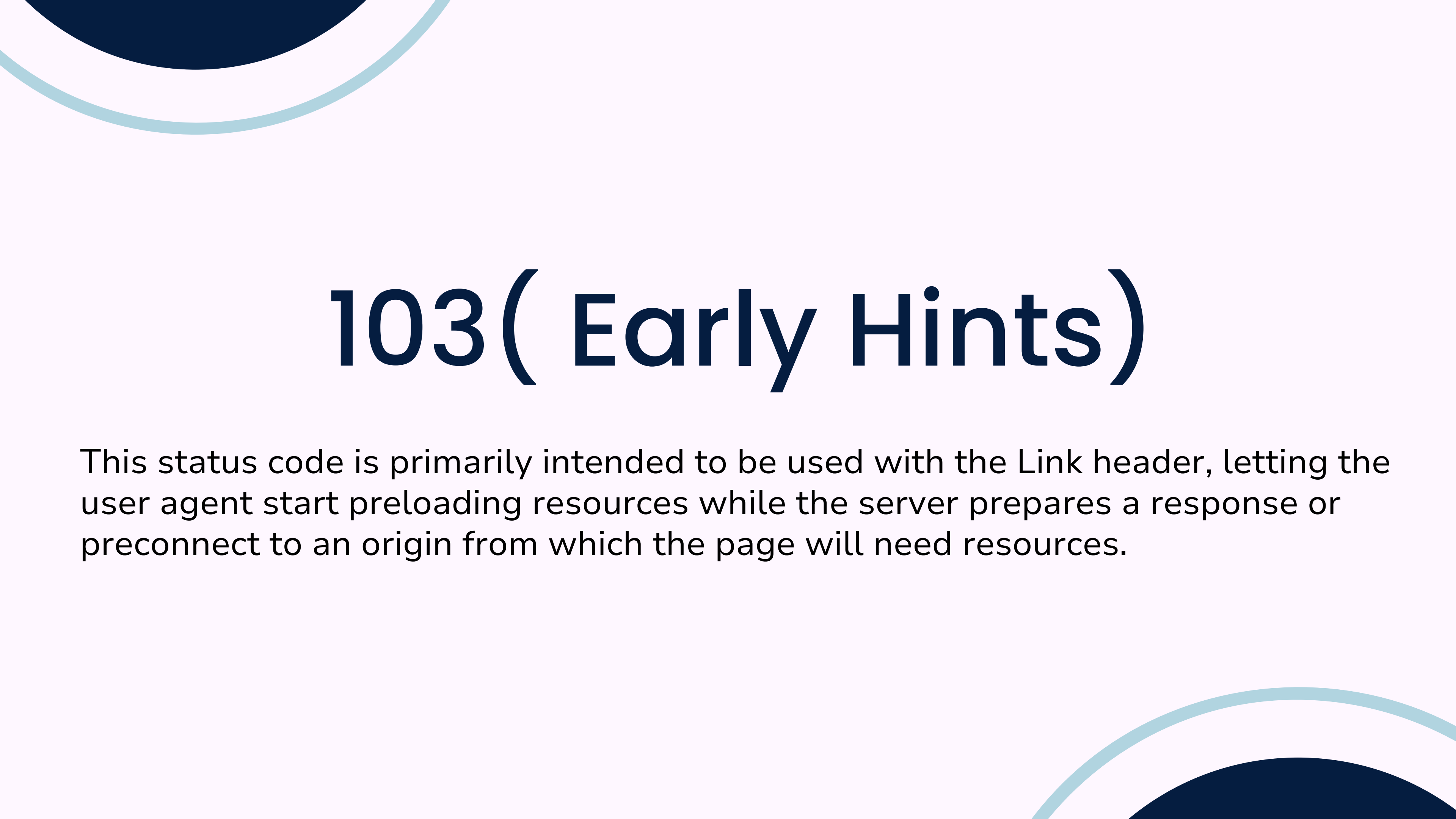
101 means “it's ok, Protocol switched”

Upgrade



102 (Processing)

indicates that a request has been received by the server, but no status was available at the time of the response.



103(Early Hints)

This status code is primarily intended to be used with the Link header, letting the user agent start preloading resources while the server prepares a response or preconnect to an origin from which the page will need resources.



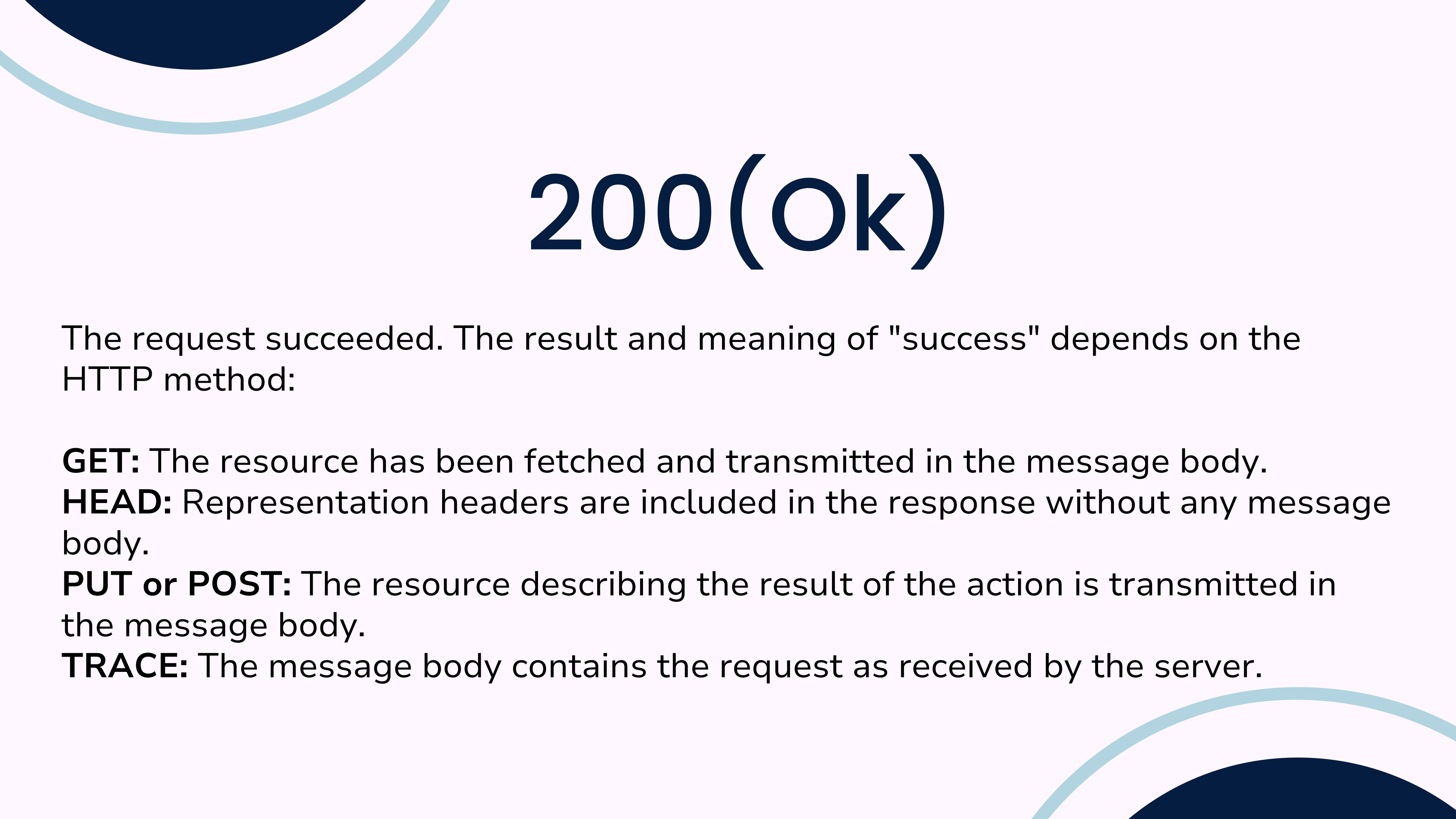
2xx: Successful responses

The 1xx response codes are used only in negotiations between an HTTP client and server

200 (Ok) - 201 (Created)

202 Accepted - 203 Non-Authoritative information

204 (No Content) - 105 (Reset Content)



200(ok)

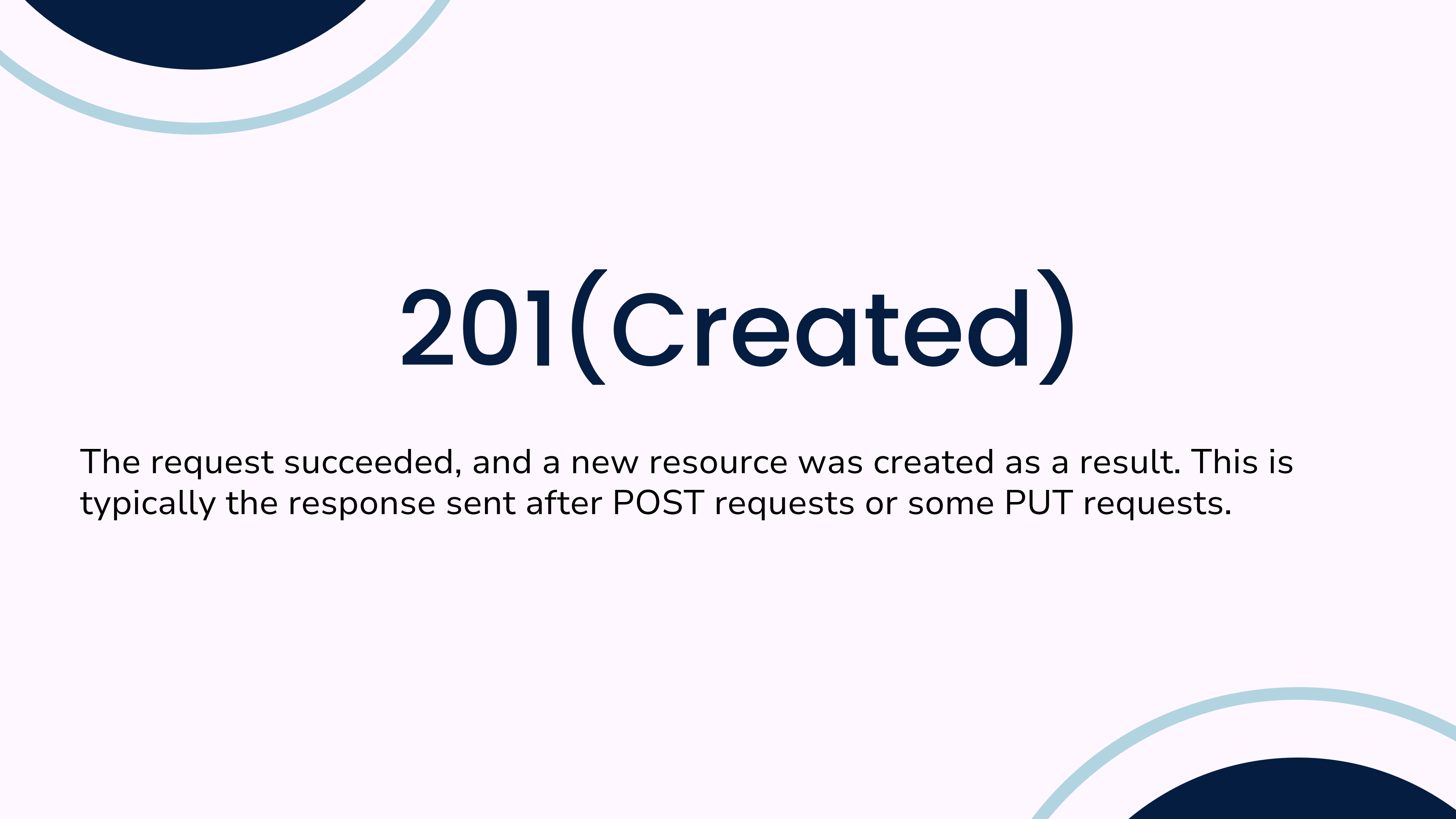
The request succeeded. The result and meaning of "success" depends on the HTTP method:

GET: The resource has been fetched and transmitted in the message body.

HEAD: Representation headers are included in the response without any message body.

PUT or POST: The resource describing the result of the action is transmitted in the message body.

TRACE: The message body contains the request as received by the server.



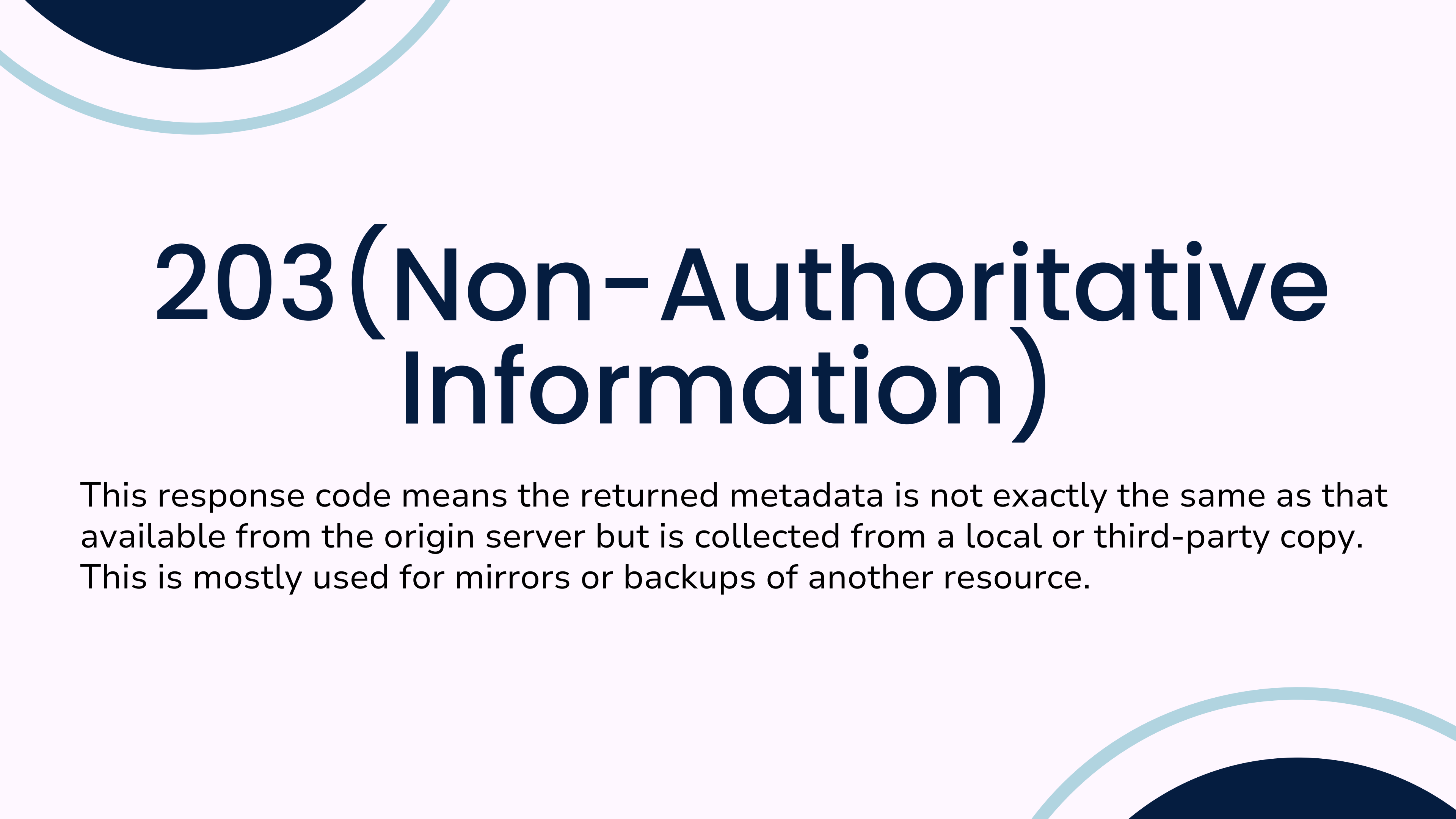
201(Created)

The request succeeded, and a new resource was created as a result. This is typically the response sent after POST requests or some PUT requests.



202(Accepted)

The request has been received but has not yet been acted upon. It is noncommittal, since there is no way in HTTP to later send an asynchronous response indicating the outcome of the request. It is intended for cases where another process or server handles the request, or for batch processing.



203 (Non-Authoritative Information)

This response code means the returned metadata is not exactly the same as that available from the origin server but is collected from a local or third-party copy. This is mostly used for mirrors or backups of another resource.



204(No Content)

There is no content to send for this request, but the headers are useful. The user agent may update its cached headers for this resource with the new ones.



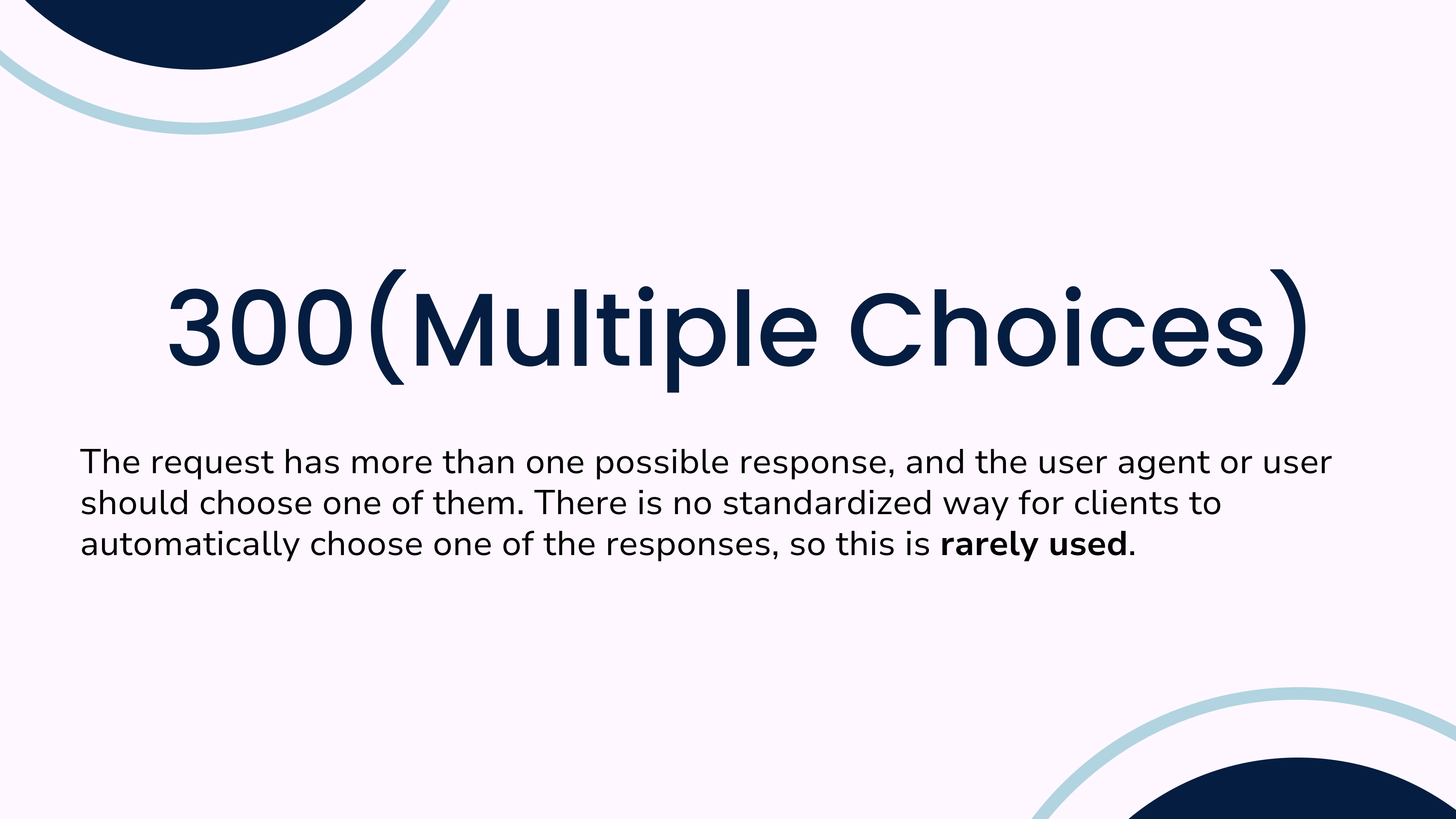
205 (Reset Content)

Tells the user agent to reset the document which sent this request.



3xx: Redirection messages

These indicate that further action is needed to complete the request, usually involving URL redirection.



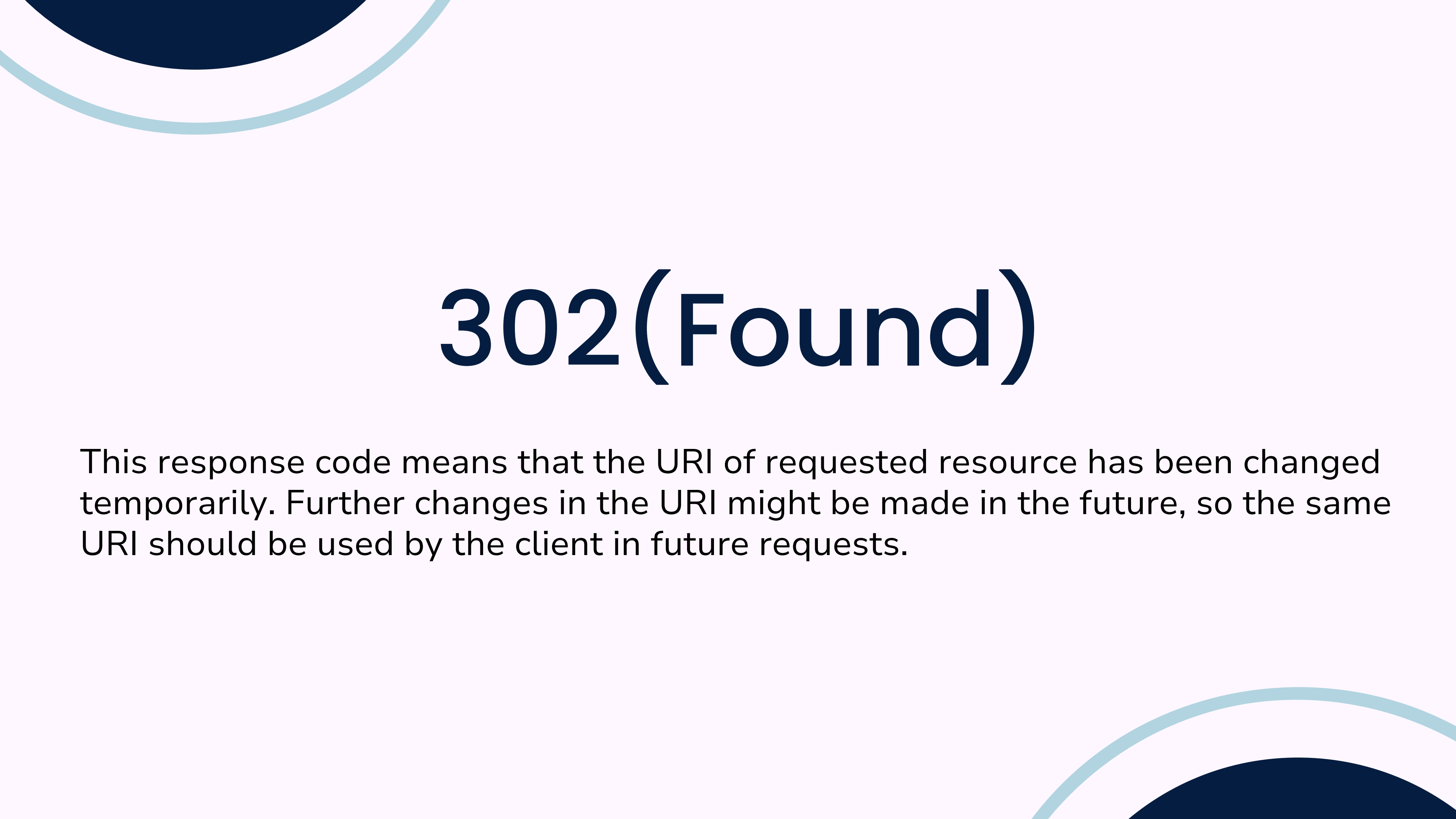
300 (Multiple Choices)

The request has more than one possible response, and the user agent or user should choose one of them. There is no standardized way for clients to automatically choose one of the responses, so this is **rarely used**.



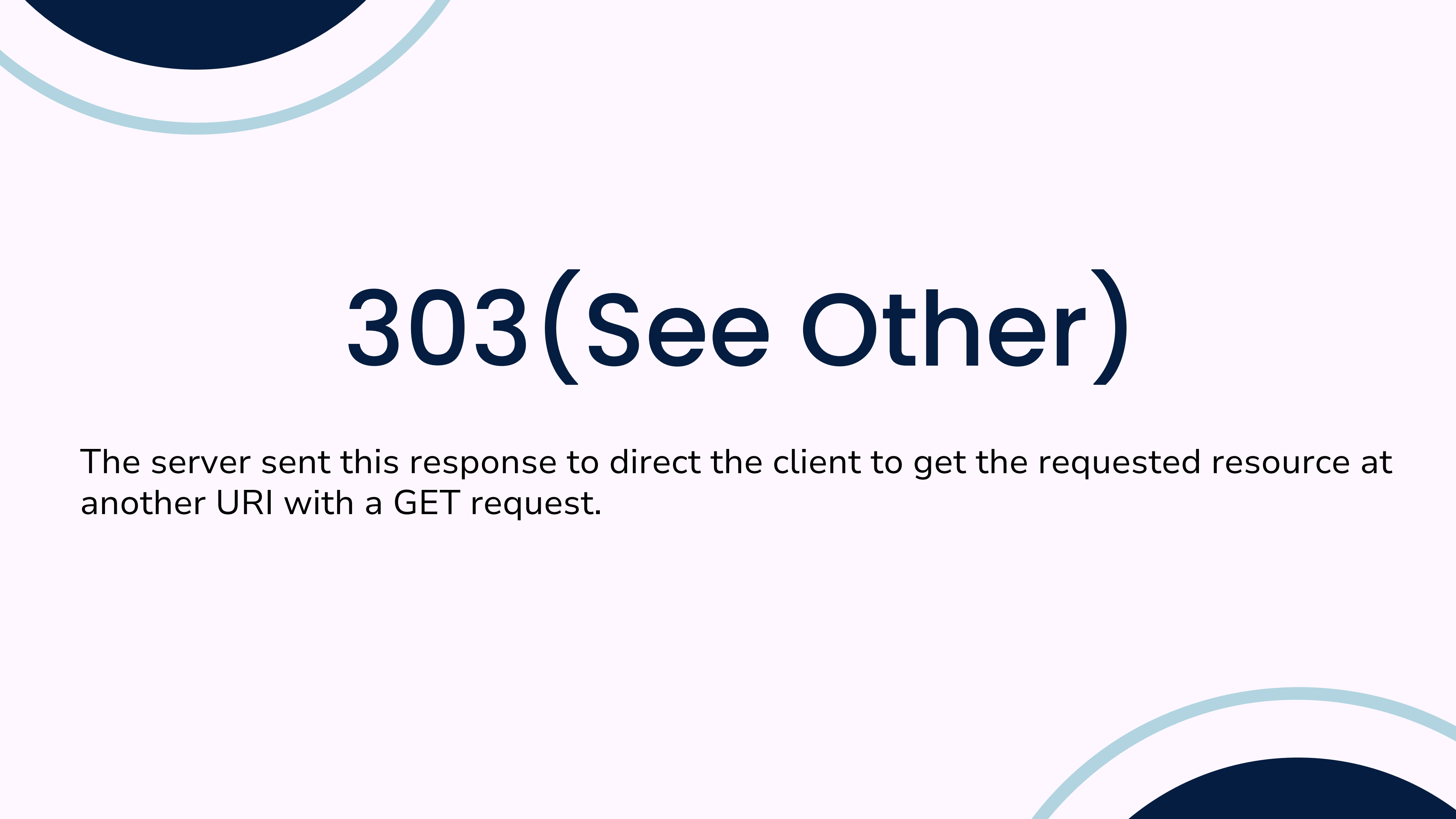
301(Moved Permanently)

The URL of the requested resource has been changed permanently.
The new URL is given in the response.



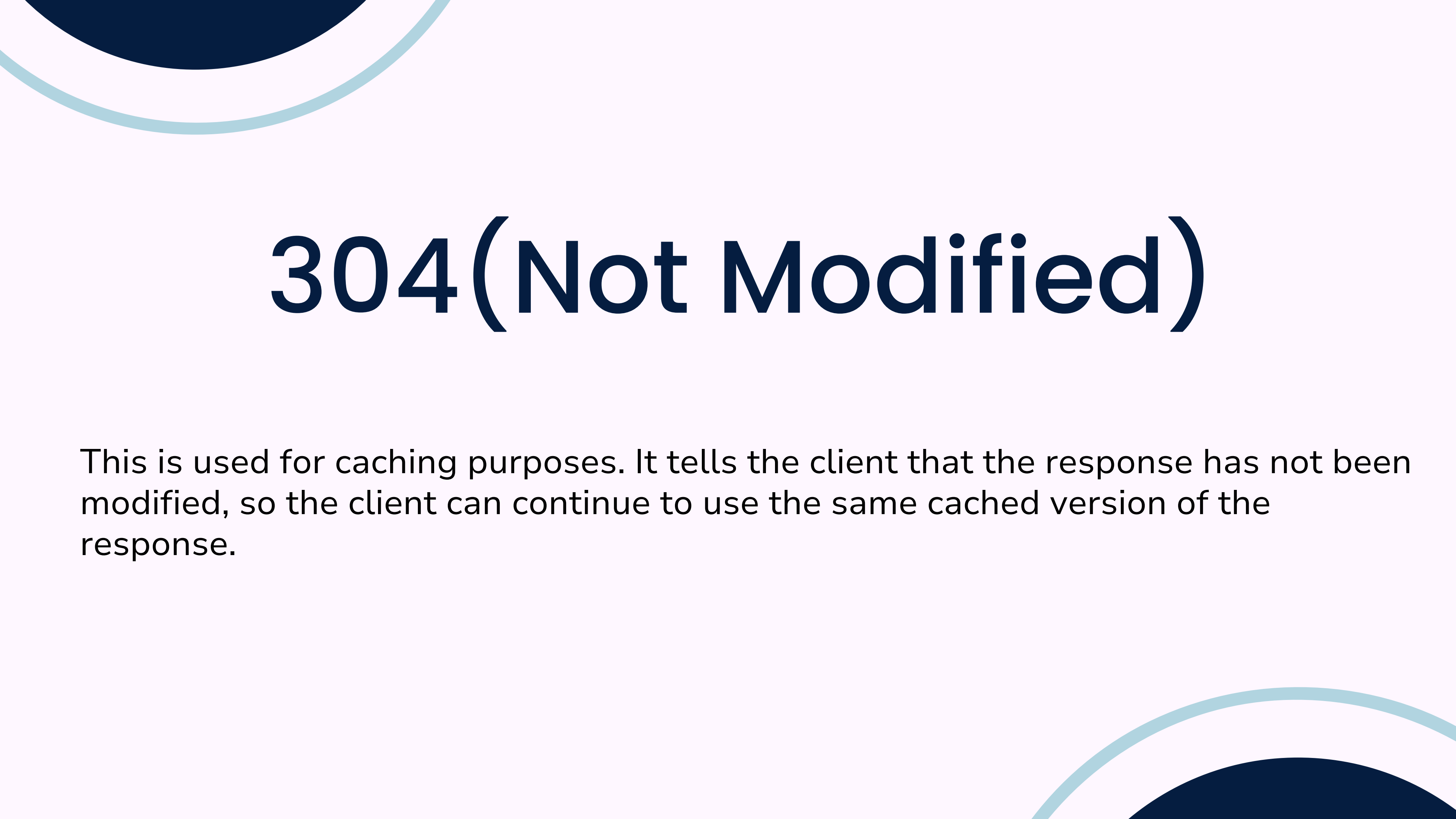
302(Found)

This response code means that the URI of requested resource has been changed temporarily. Further changes in the URI might be made in the future, so the same URI should be used by the client in future requests.



303 (See Other)

The server sent this response to direct the client to get the requested resource at another URI with a GET request.



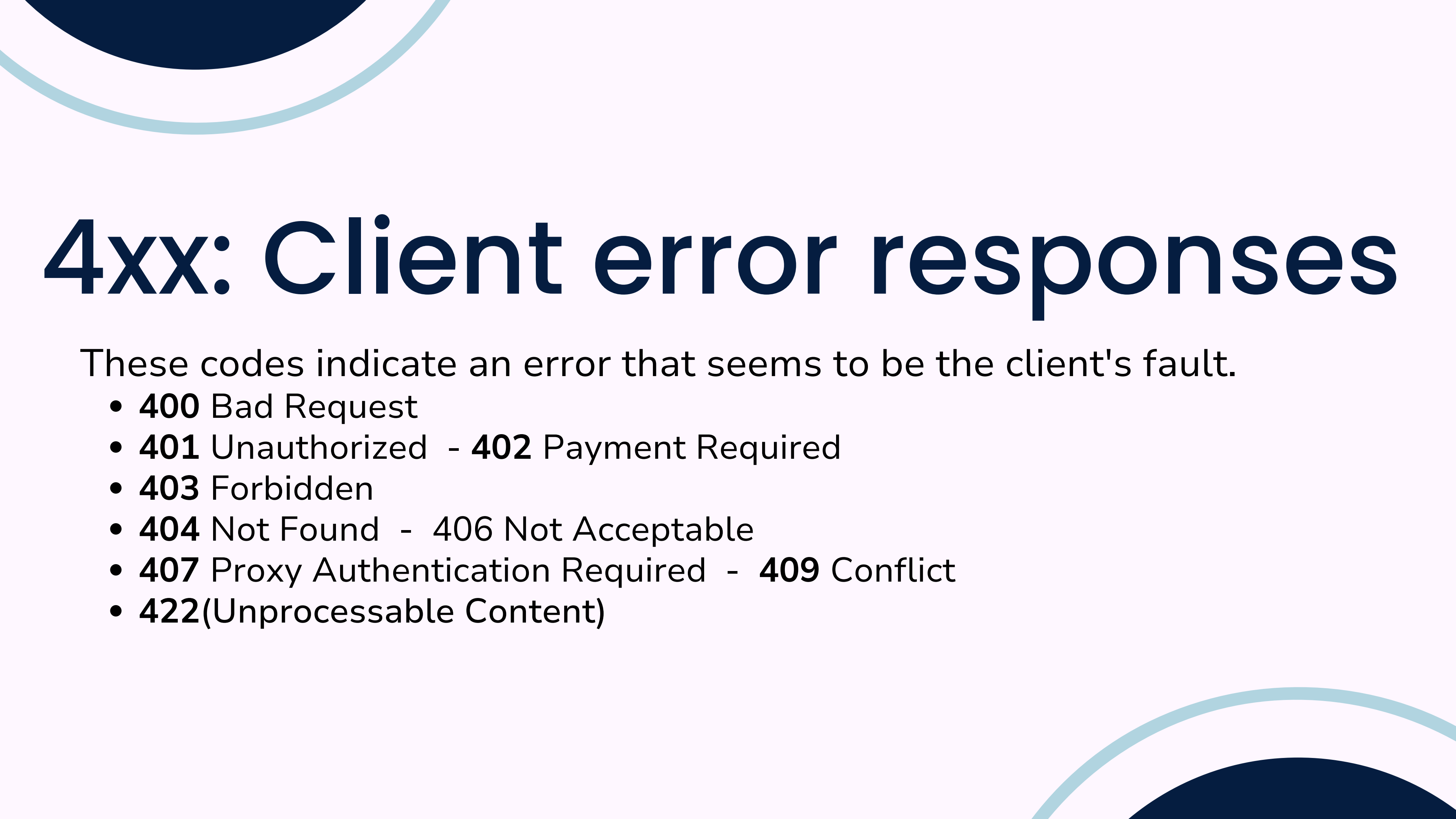
304 (Not Modified)

This is used for caching purposes. It tells the client that the response has not been modified, so the client can continue to use the same cached version of the response.



308 (Permanent Redirect)

This means that the resource is now permanently located at another URI, specified by the Location response header.



4xx: Client error responses

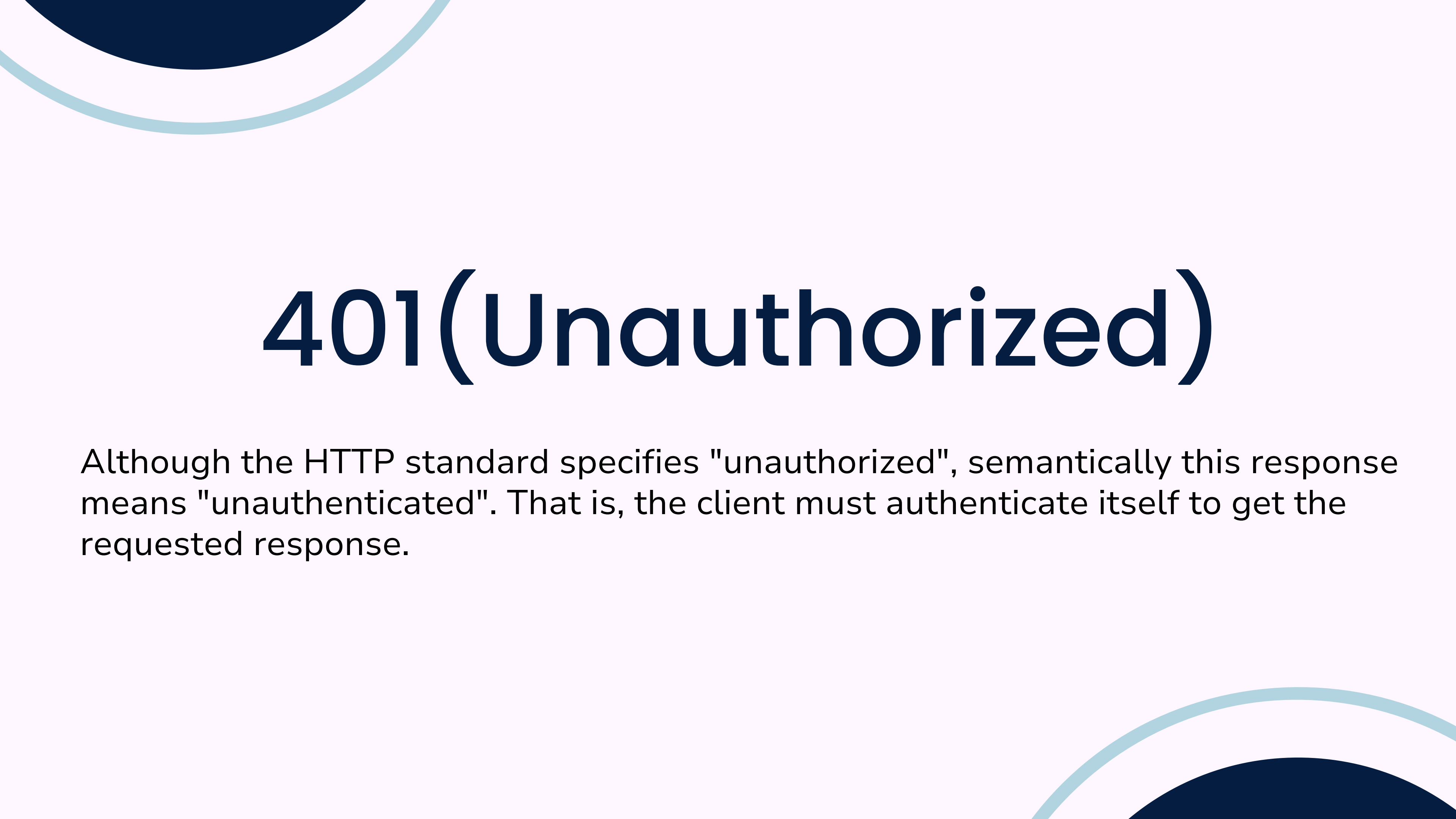
These codes indicate an error that seems to be the client's fault.

- **400** Bad Request
- **401** Unauthorized - **402** Payment Required
- **403** Forbidden
- **404** Not Found - **406** Not Acceptable
- **407** Proxy Authentication Required - **409** Conflict
- **422**(Unprocessable Content)



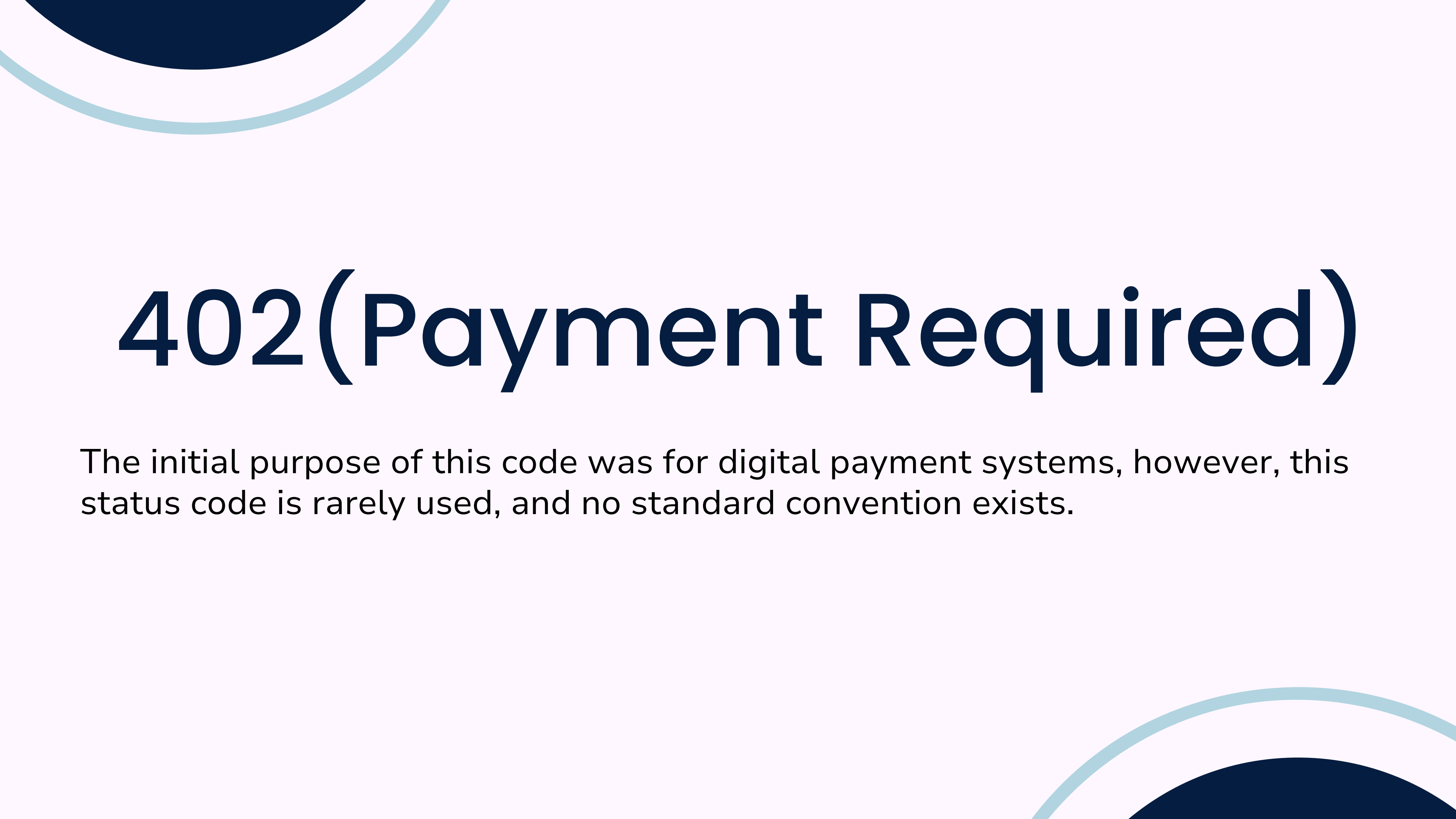
400 (Bad Request)

The server cannot or will not process the request due to something that is perceived to be a client error (e.g., malformed request syntax, invalid request message framing, or deceptive request routing).



401(Unauthorized)

Although the HTTP standard specifies "unauthorized", semantically this response means "unauthenticated". That is, the client must authenticate itself to get the requested response.



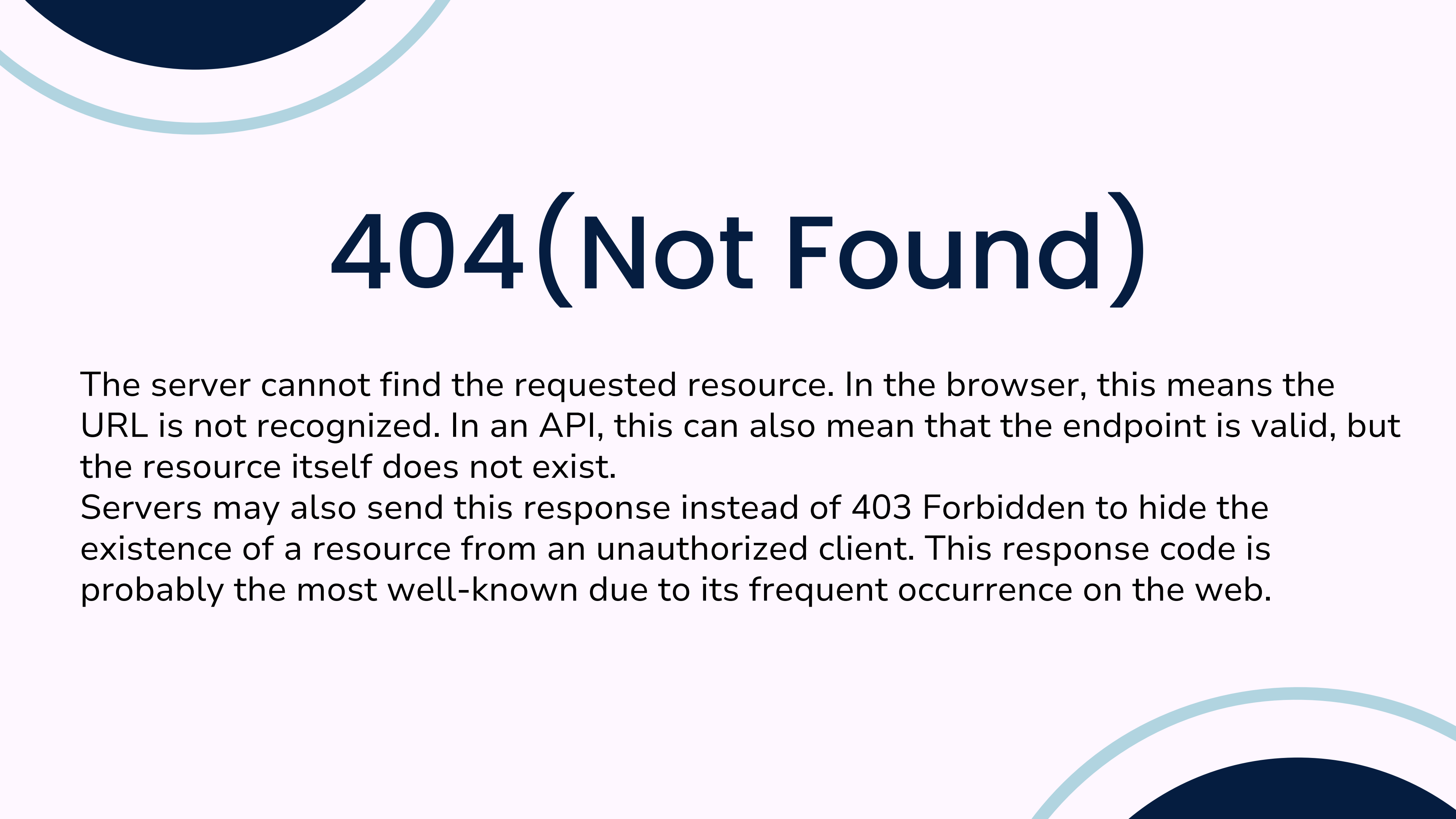
402(Payment Required)

The initial purpose of this code was for digital payment systems, however, this status code is rarely used, and no standard convention exists.



403 (Forbidden)

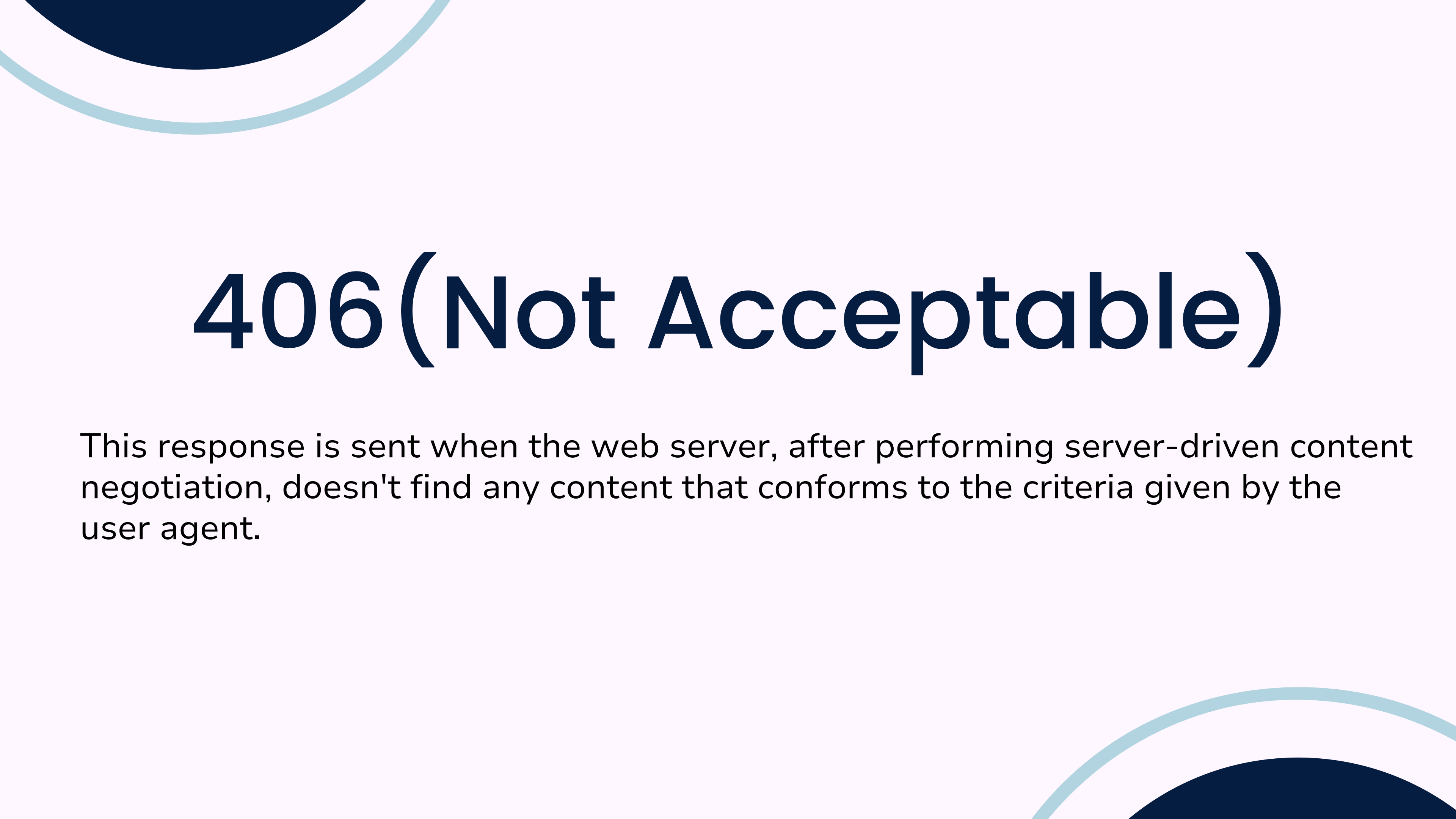
The client does not have access rights to the content; that is, it is unauthorized, so the server is refusing to give the requested resource. Unlike **401 Unauthorized**, the client's identity is known to the server.



404 (Not Found)

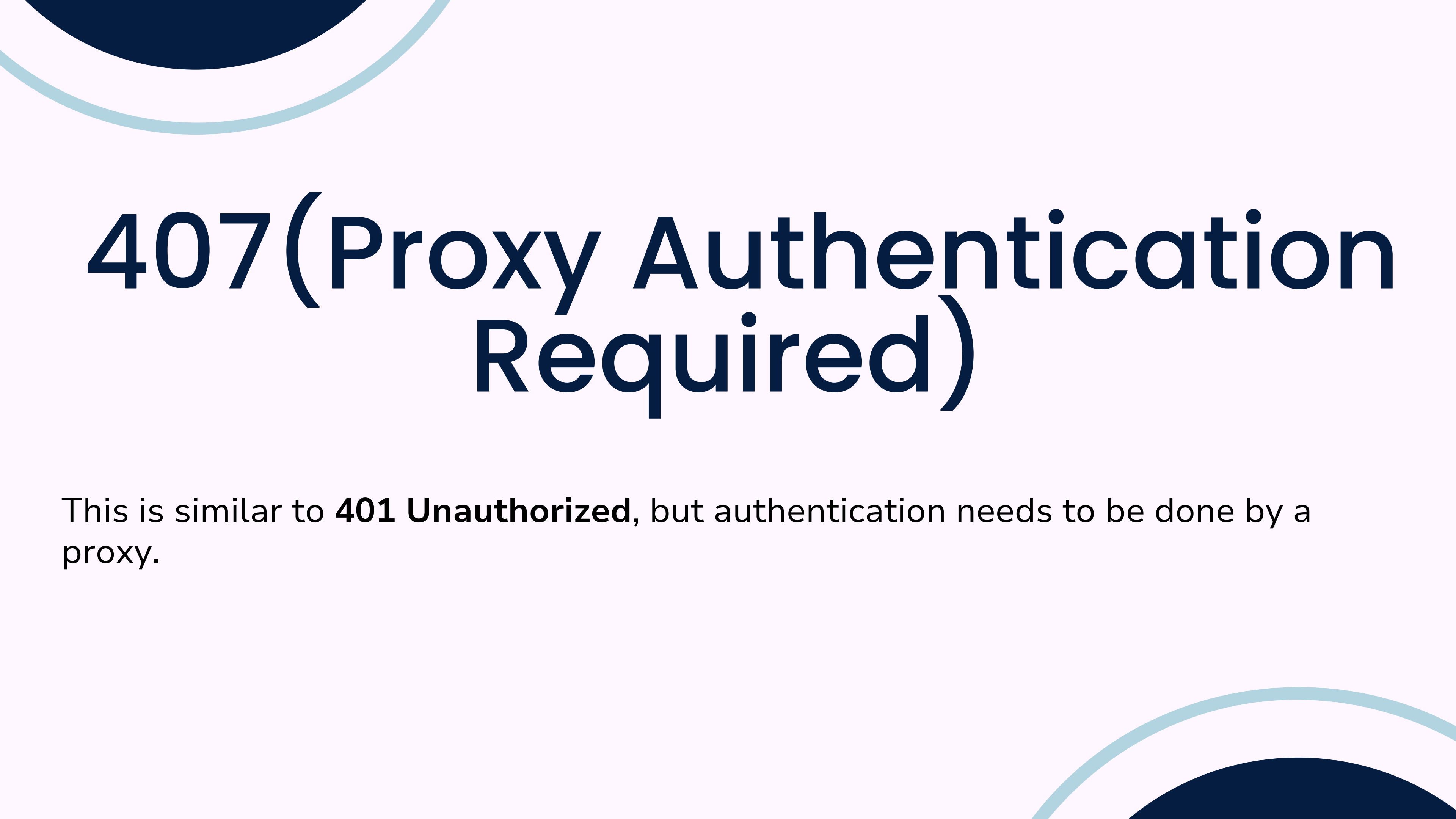
The server cannot find the requested resource. In the browser, this means the URL is not recognized. In an API, this can also mean that the endpoint is valid, but the resource itself does not exist.

Servers may also send this response instead of 403 Forbidden to hide the existence of a resource from an unauthorized client. This response code is probably the most well-known due to its frequent occurrence on the web.



406 (Not Acceptable)

This response is sent when the web server, after performing server-driven content negotiation, doesn't find any content that conforms to the criteria given by the user agent.



407 (Proxy Authentication Required)

This is similar to **401 Unauthorized**, but authentication needs to be done by a proxy.



409 (Conflict)

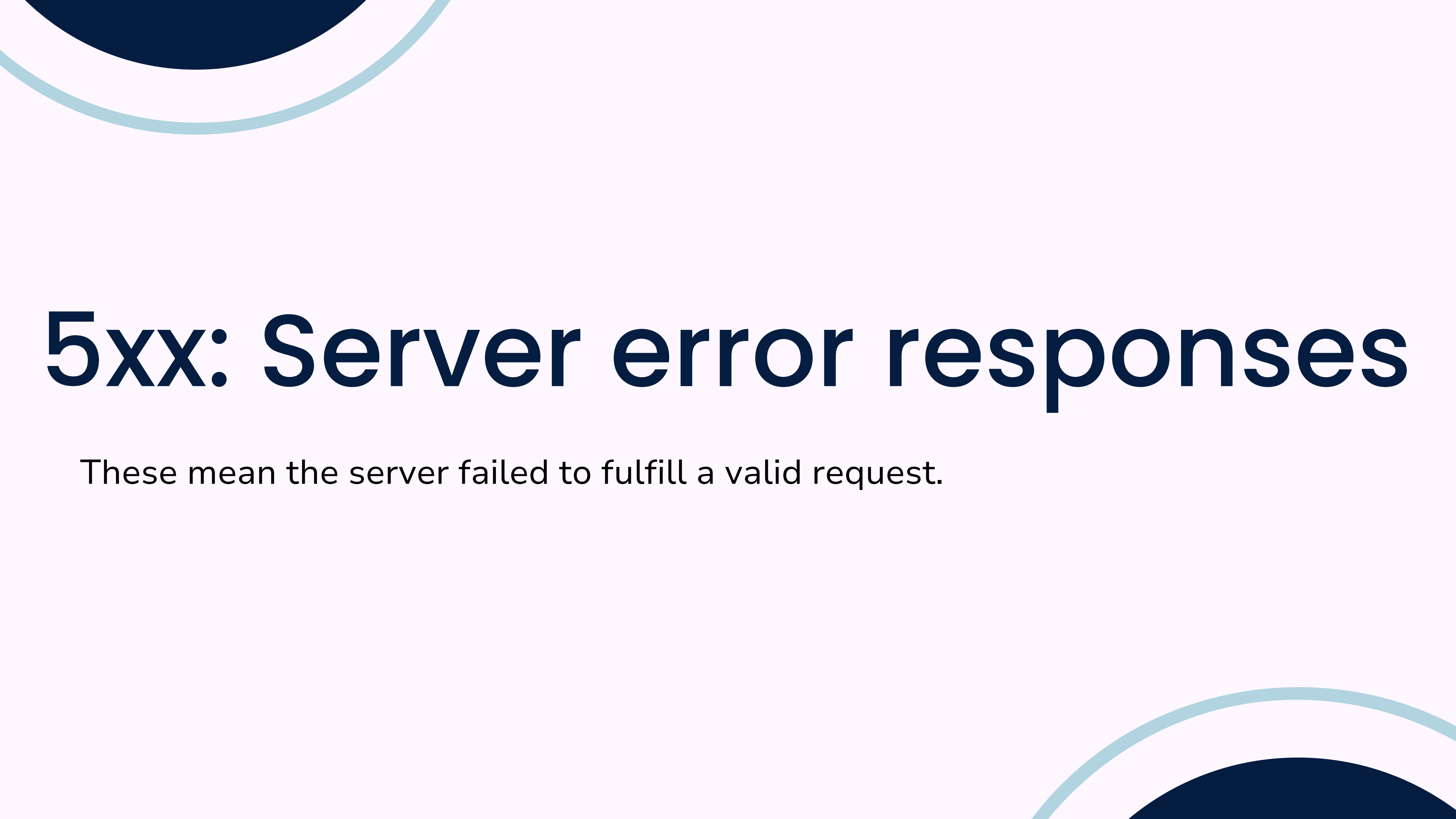
This response is sent when a request conflicts with the current state of the server.



422(Unprocessable Content)

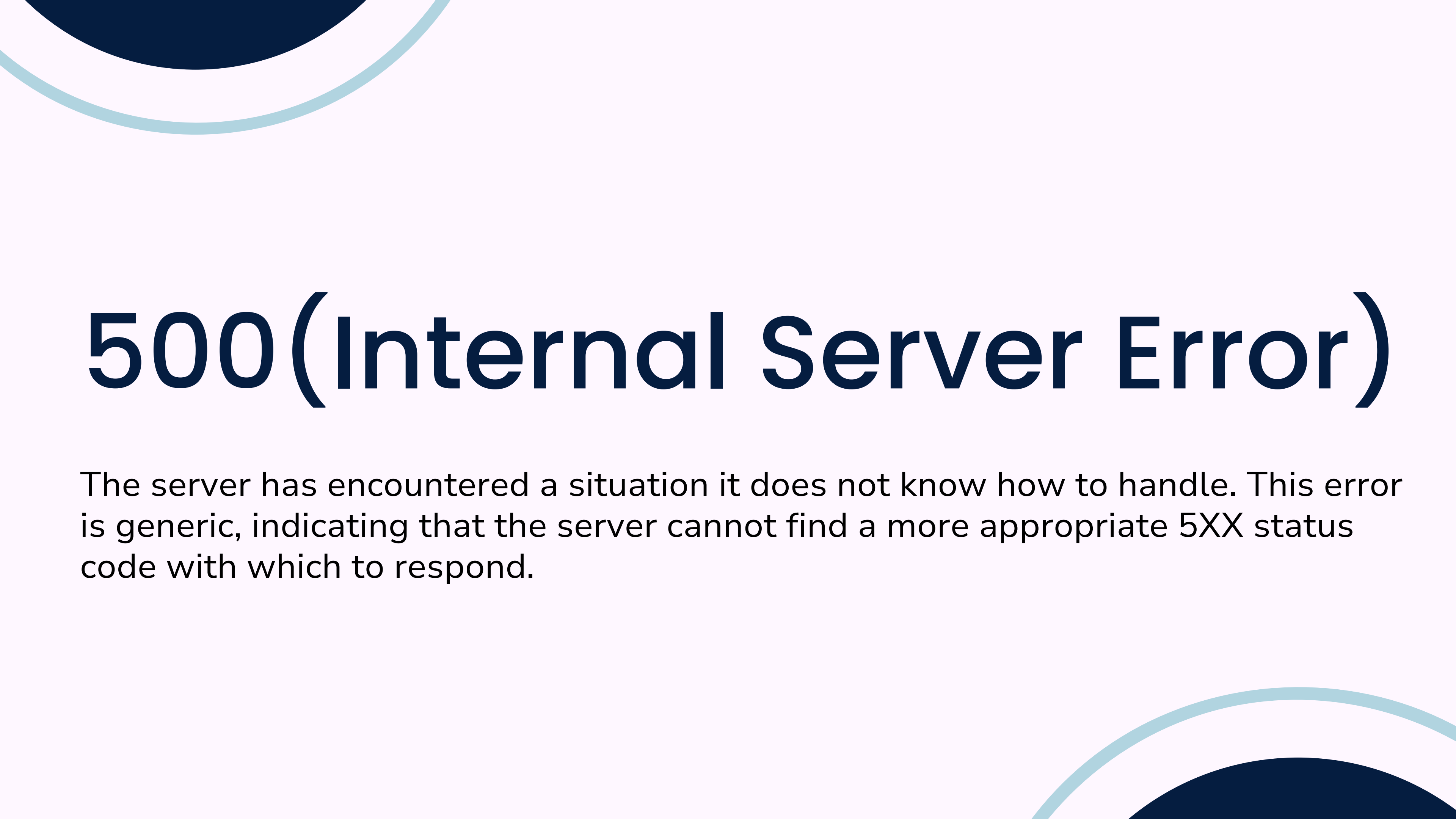
The request was well-formed but was unable to be followed due to semantic errors.





5xx: Server error responses

These mean the server failed to fulfill a valid request.



500 (Internal Server Error)

The server has encountered a situation it does not know how to handle. This error is generic, indicating that the server cannot find a more appropriate 5XX status code with which to respond.



501 (Not Implemented)

The request method is not supported by the server and cannot be handled. The only methods that servers are required to support (and therefore that must not return this code) are **GET** and **HEAD**.



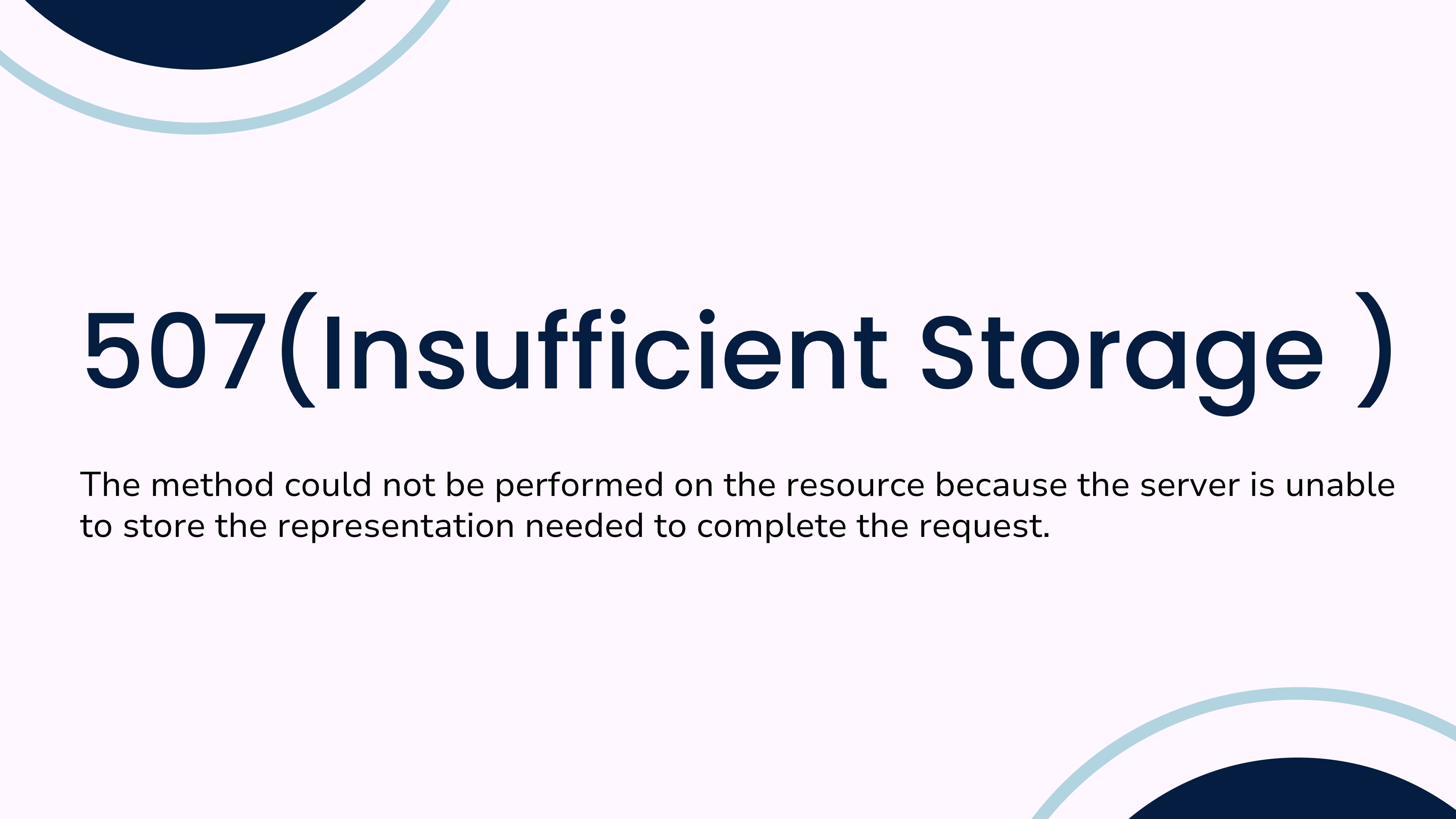
502 (Bad Gateway)

This error response means that the server, while working as a gateway to get a response needed to handle the request, got an invalid response.



504 (Gateway Timeout)

This error response is given when the server is acting as a gateway and cannot get a response in time.



507 (Insufficient Storage)

The method could not be performed on the resource because the server is unable to store the representation needed to complete the request.



504 (Gateway Timeout)

This error response is given when the server is acting as a gateway and cannot get a response in time.



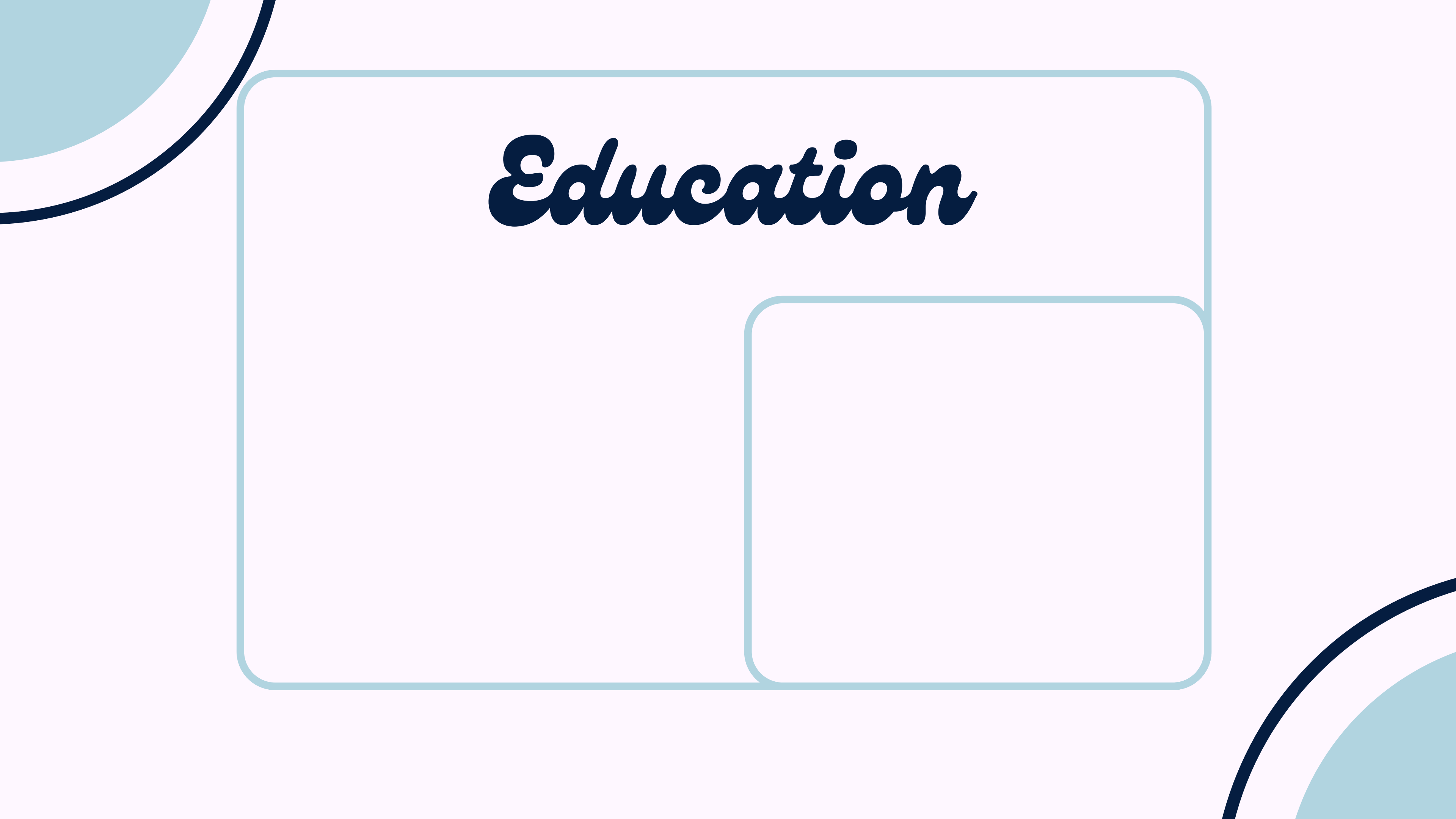
504 (Gateway Timeout)

This error response is given when the server is acting as a gateway and cannot get a response in time.

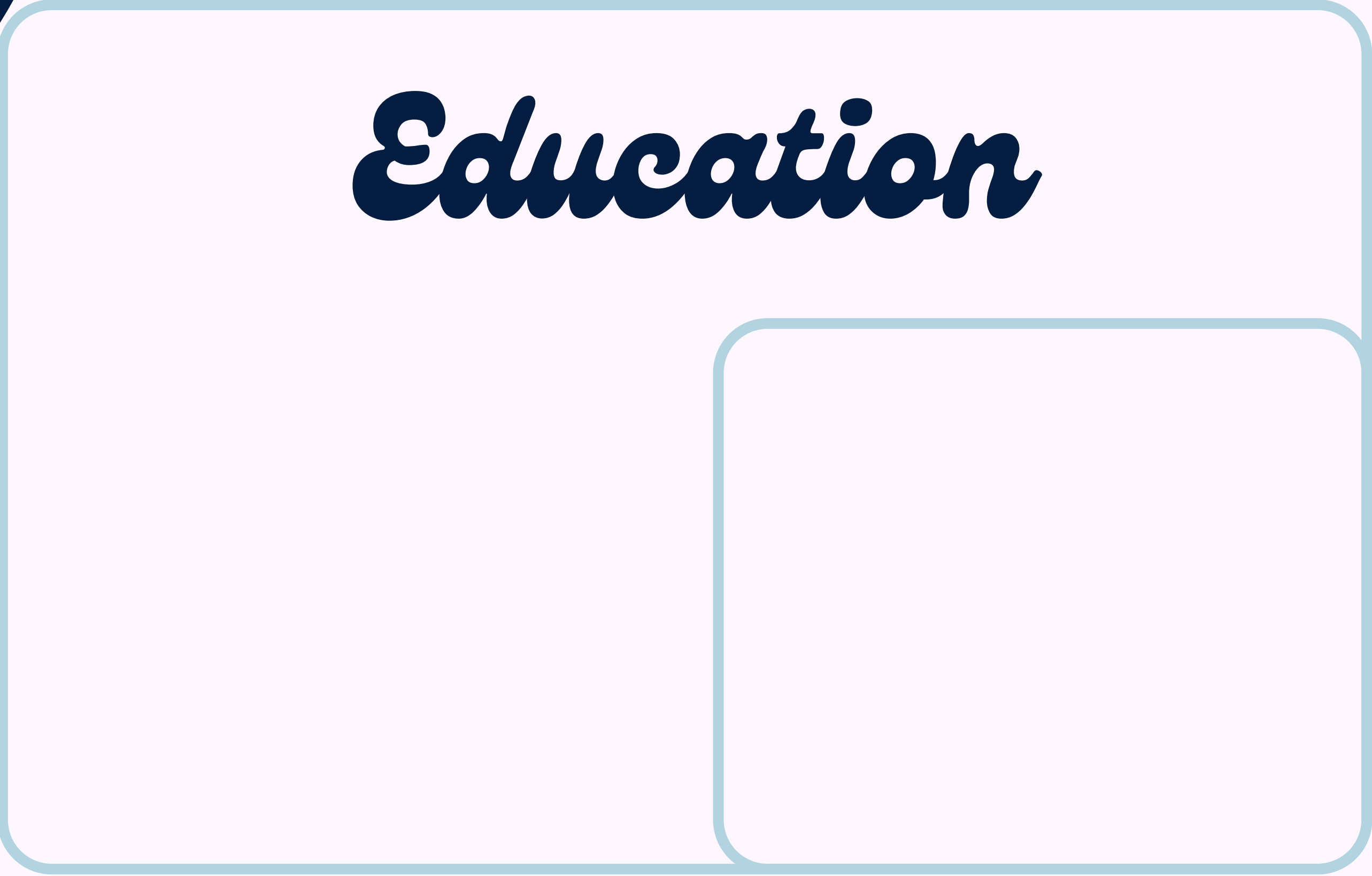
Our Mission

Lorem ipsum dolor sit
amet, consectetur
adipiscing elit.
Vestibulum ac ligula
ex. Ut gravida orci
non eros rhoncus, nec
venenatis orci
malesuada.

Lorem ipsum dolor sit
amet, consectetur
adipiscing elit.
Vestibulum ac ligula
ex. Ut gravida orci
non eros rhoncus, nec
venenatis orci
malesuada.



Education



Work Experience

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.



My Portfolio

Project 1

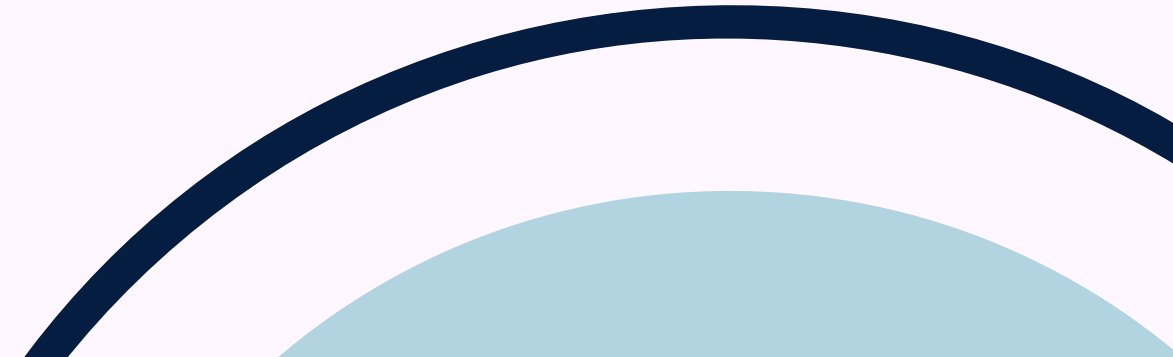
Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Vestibulum ac ligula ex. Ut
gravida orci non eros rhoncus,
nec venenatis orci malesuada.

Project 2

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Vestibulum ac ligula ex. Ut
gravida orci non eros rhoncus,
nec venenatis orci malesuada.

Project 3

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Vestibulum ac ligula ex. Ut
gravida orci non eros rhoncus,
nec venenatis orci malesuada.





*Thank
You*